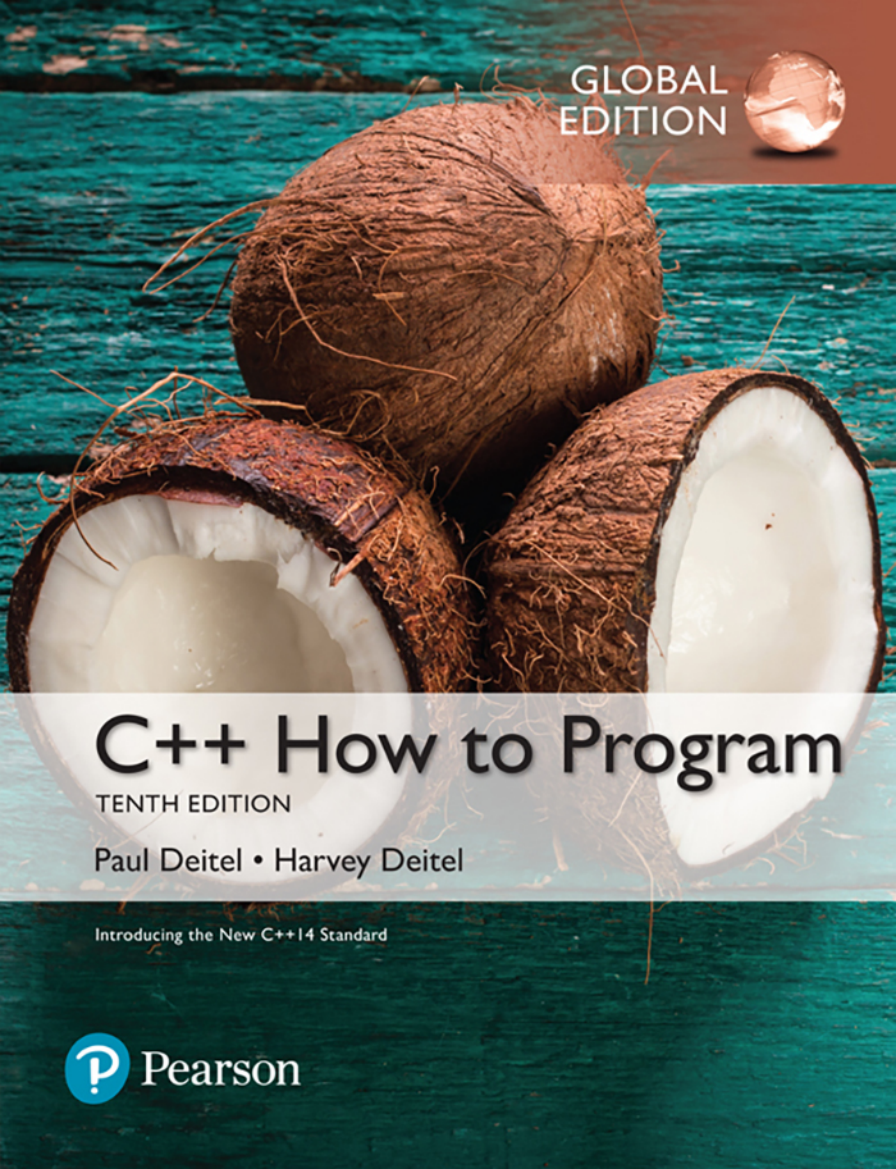


C++ How to Program

TENTH EDITION

Paul Deitel • Harvey Deitel

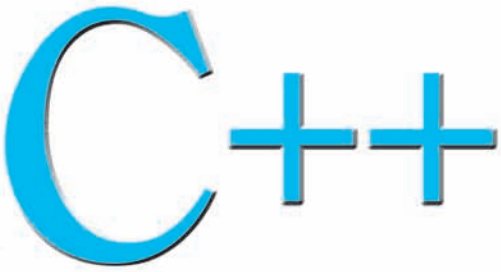
Introducing the New C++14 Standard

Pearson

C++

HOW TO PROGRAM

This page intentionally left blank



HOW TO PROGRAM

Introducing
the New C++14
Standard

TENTH EDITION
GLOBAL EDITION

Paul Deitel
Deitel & Associates, Inc.

Harvey Deitel
Deitel & Associates, Inc.

PEARSON

Boston Columbus Hoboken Indianapolis New York San Francisco
Amsterdam Cape Town Dubai London Madrid Milan Munich Paris Montreal
Toronto Delhi Mexico City São Paulo Sydney Hong Kong Seoul Singapore Taipei Tokyo

Vice President, Editorial Director: ***Marcia Horton***
Acquisitions Editor: ***Tracy Johnson***
Editorial Assistant: ***Kristy Alaura***
Acquisitions Editor, Global Editions: ***Sourabh Maheshwari***
VP of Marketing: ***Christy Lesko***
Director of Field Marketing: ***Tim Galligan***
Product Marketing Manager: ***Bram Van Kempen***
Field Marketing Manager: ***Demetrius Hall***
Marketing Assistant: ***Jon Bryant***
Director of Product Management: ***Erin Gregg***
Team Lead, Program and Project Management: ***Scott Disanno***
Program Manager: ***Carole Snyder***
Project Manager: ***Robert Engelhardt***
Project Editor, Global Editions: ***K.K. Neelakantan***
Senior Manufacturing Controller, Global Editions: ***Trudy Kimber***
Senior Specialist, Program Planning and Support: ***Maura Zaldivar-Garcia***
Media Production Manager, Global Editions: ***Vikram Kumar***
Cover Art: ***Finevector / Shutterstock***
Cover Design: ***Lumina Datamatics***
R&P Manager: ***Rachel Youdelman***
R&P Project Manager: ***Timothy Nicholls***
Inventory Manager: ***Meredith Maresca***

Credits and acknowledgments borrowed from other sources and reproduced, with permission, in this textbook appear on page 6.

Pearson Education Limited
Edinburgh Gate
Harlow
Essex CM20 2JE
England

and Associated Companies throughout the world

Visit us on the World Wide Web at:
www.pearsonglobaleditions.com

British Library Cataloguing-in-Publication Data

A catalogue record for this book is available from the British Library

10 9 8 7 6 5 4 3 2 1

ISBN 10: 1-292-15334-2
ISBN 13: 978-1-292-15334-6

Typeset by GEX Publishing Services

Printed and bound in Malaysia

In memory of Marvin Minsky,
a founding father of the
field of artificial intelligence.

It was a privilege to be your student in two graduate courses at M.I.T. Every lecture you gave inspired your students to think beyond limits.

Harvey Deitel

Trademarks

DEITEL and the double-thumbs-up bug are registered trademarks of Deitel and Associates, Inc.

Carnegie Mellon Software Engineering Institute™ is a trademark of Carnegie Mellon University.

CERT® is registered in the U.S. Patent and Trademark Office by Carnegie Mellon University.

UNIX is a registered trademark of The Open Group.

Microsoft and/or its respective suppliers make no representations about the suitability of the information contained in the documents and related graphics published as part of the services for any purpose. All such documents and related graphics are provided "as is" without warranty of any kind. Microsoft and/or its respective suppliers hereby disclaim all warranties and conditions with regard to this information, including all warranties and conditions of merchantability, whether express, implied or statutory, fitness for a particular purpose, title and non-infringement. In no event shall Microsoft and/or its respective suppliers be liable for any special, indirect or consequential damages or any damages whatsoever resulting from loss of use, data or profits, whether in an action of contract, negligence or other tortious action, arising out of or in connection with the use or performance of information available from the services.

The documents and related graphics contained herein could include technical inaccuracies or typographical errors. Changes are periodically added to the information herein. Microsoft and/or its respective suppliers may make improvements and/or changes in the product(s) and/or the program(s) described herein at any time. Partial screen shots may be viewed in full within the software version specified.

Microsoft® and Windows® are registered trademarks of the Microsoft Corporation in the U.S.A. and other countries. Screen shots and icons reprinted with permission from the Microsoft Corporation. This book is not sponsored or endorsed by or affiliated with the Microsoft Corporation.

Throughout this book, trademarks are used. Rather than put a trademark symbol in every occurrence of a trademarked name, we state that we are using the names in an editorial fashion only and to the benefit of the trademark owner, with no intention of infringement of the trademark.

Contents

Chapters 23–26 and Appendices F–J are PDF documents posted online at the book's Companion Website, which is accessible from

```
http://www.pearsonglobaleditions.com/deitel
```

See the inside front cover for more information.

Chapters 23–26 and Appendices F–J are PDF documents posted online at the book's Companion Website, which is accessible from

```
http://www.pearsonglobaleditions.com/deitel
```

See the inside front cover for more information.

Preface

Welcome to the C++ computer programming language and *C++ How to Program, Tenth Edition*. We believe that this book and its support materials will give you an informative, challenging and entertaining introduction to C++. The book presents leading-edge computing technologies in a friendly manner appropriate for introductory college course sequences, based on the curriculum recommendations of two key professional organizations—the ACM and the IEEE.[1]

If you haven't already done so, please read the back cover and check out the additional reviewer comments on the inside back cover and the facing page—these capture the essence of the book concisely. In this Preface we provide more detail for students, instructors and professionals.

At the heart of the book is the Deitel signature ***live-code approach***—we present most concepts in the context of complete working programs followed by sample executions, rather than in code snippets. Read the **Before You Begin** section to learn how to set up your Linux-based, Windows-based or Apple OS X-based computer to run the hundreds of code examples. All the source code is available at

```
http://www.pearsonglobaleditions.com/deitel
```

Use the source code we provide to run each program as you study it.

Contacting the Authors

As you read the book, if you have questions, we're easy to reach at

```
deitel@deitel.com
```

We'll respond promptly. For book updates, visit

```
http://www.deitel.com/books/cpphtp10
```

Join the Deitel & Associates, Inc. Social Media Communities

Join the Deitel social media communities on

- Facebook®—`http://facebook.com/DeitelFan`
- LinkedIn®—`http://bit.ly/DeitelLinkedIn`

1. *Computer Science Curricula 2013 Curriculum Guidelines for Undergraduate Degree Programs in Computer Science*, December 20, 2013, The Joint Task Force on Computing Curricula, Association for Computing Machinery (ACM), IEEE Computer Society.

- Twitter®—`http://twitter.com/deitel`
- Google+™—`http://google.com/+DeitelFan`
- YouTube®—`http://youtube.com/DeitelTV`

and subscribe to the *Deitel® Buzz Online* newsletter

```
http://www.deitel.com/newsletter/subscribe.html
```

The C++11 and C++14 Standards

These are exciting times in the programming languages community with each of the major languages striving to keep pace with compelling new programming technologies. In the three decades of C++'s development prior to 2011, only a few new versions of the language were released. Now the ISO C++ Standards Committee is committed to releasing a new standard every three years and the compiler vendors are building in the new features promptly. *C++ How to Program, 10/e* is based on the **C++11** and **C++14** standards published in 2011 and 2014, respectively. C++17 is already under active development. Throughout the book, C++11 and C++14 features are marked with the "11" and "14" icons, respectively, that you see here in the margin. Fig. 1 lists the book's first references to the 77 C++11 and C++14 features we discuss.

11
14

C++11 and C++14 features in *C++ How to Program, 10/e*

Chapter 3
In-class initializers

Chapter 4
Keywords new in C++11

Chapter 5
`long long int` type

Chapter 6
Non-deterministic random number generation
Scoped `enum`s
Specifying the type of an `enum`'s constants
Unsigned `long long int`
Using `'` to separate groups of digits in a numeric literals (**C++14**)

Chapter 7
`array` container
`auto` for type inference
List initializing a `vector`
Range-based `for` statement

Chapter 8
`begin`/`end` functions
`nullptr`

Chapter 9
Delegating constructors

Chapter 10
`deleted` member functions
`explicit` conversion operators
List initializing a dynamically allocated array
List initializers in constructor calls
`string` object literals (**C++14**)

Chapter 11
`final` classes
`final` member functions
Inheriting base-class constructors

Chapter 12
`defaulted` member functions
`override` keyword

Chapter 13
`operator bool` for streams

Chapter 14
`quoted` stream manipulator (**C++14**)
`string` objects for file names

Chapter 15
`cbegin`/`cend` container member functions
Compiler fix for `>>` in template types
`crbegin`/`crend` container member functions
`forward_list` container
Global functions `cbegin`/`cend`, `rbegin`/`rend` and `crbegin`/`crend` (**C++14**)
Heterogeneous lookup in associative containers (**C++14**)
Immutable keys in associative containers

Fig. 1 | First references to C++11 and C++14 features in *C++ How to Program, 10/e*. (Part 1 of 2.)

C++11 and C++14 features in *C++ How to Program, 10/e*

Chapter 15 (cont.)
`insert` container member functions return iterators
List initialization of key–value pairs
List initialization of `pairs`
Return value list initialization
`shrink_to_fit` `vector`/`deque` member function

Chapter 16
`all_of` algorithm
`any_of` algorithm
`copy_if` algorithm
`copy_n` algorithm
`equal` algorithm that accepts two ranges (C++14)
`find_if_not` algorithm
Generic lambdas (C++14)
Lambda expressions
`min` and `max` algorithms with `initializer_list` parameters
`minmax` algorithm

Chapter 16 (cont.)
`minmax_element` algorithm
`mismatch` algorithm that accepts two ranges (C++14)
`none_of` algorithm
`random_shuffle` is deprecated (C++14)—replaced with `shuffle` and C++11 random-number generation
`swap` non-member function

Chapter 17
`make_unique` to create a `unique_ptr` (C++14)
`noexcept`
`unique_ptr` smart pointer

Chapter 18
Default type arguments in function templates

Chapter 21
Numeric conversion functions

Chapter 22
Binary literals (C++14)

Chapter 24
Aggregate member initialization (C++14)
`auto` and `decltype(auto)` on return types(C++14)
`constexpr` updated(C++14)
`decltype`
`move` algorithm
Move assignment operators
`move_backward` algorithm
Move constructors
Regular expressions
Rvalue references
`shared_ptr` smart pointer
`static_assert` objects for file names
Trailing return types `for` functions
`tuple` variadic template
`tuple` addressing via type (C++14)
`weak_ptr` smart pointer

Fig. 1 | First references to C++11 and C++14 features in *C++ How to Program, 10/e*. (Part 2 of 2.)

Key Features of C++ How to Program, 10/e

- ***Conforms to the C++11 standard and the new C++14 standard.***
- ***Code thoroughly tested on three popular industrial-strength C++14 compilers.*** We tested the code examples on GNU™ C++ 5.2.1, Microsoft® Visual Studio® 2015 Community edition and Apple® Clang/LLVM in Xcode® 7.
- ***Smart pointers.*** Smart pointers help you avoid dynamic memory management errors by providing additional functionality beyond that of built-in pointers. We discuss `unique_ptr` in Chapter 17, and `shared_ptr` and `weak_ptr` in Chapter 24.
- ***Early coverage of Standard Library containers, iterators and algorithms, enhanced with C++11 and C++14 capabilities.*** The treatment of Standard Library containers, iterators and algorithms in Chapters 15 and 16 has been enhanced with additional C++11 and C++14 features. The vast majority of your data structure needs can be fulfilled by *reusing* these Standard Library capabilities. We'll show you how to build your own *custom* data structures in Chapter 19.
- ***Online Chapter 24,* C++11 and C++14 Additional Topics.** This chapter includes discussions of regular expressions, `shared_ptr` and `weak_ptr` smart pointers, move semantics, multithreading, `tuples`, `decltype`, `constexpr` and more (see Fig. 1).

- ***Random-number generation, simulation and game playing.*** To help make programs more secure, we include a treatment of C++11's non-deterministic random-number generation capabilities.
- ***Pointers.*** We provide thorough coverage of the built-in pointer capabilities and the intimate relationship among built-in pointers, C strings and built-in arrays.
- ***Visual presentation of searching and sorting, with a simple explanation of Big O.***
- ***Printed book contains core content; additional content is online.*** Several online chapters and appendices are included. These are available in searchable PDF format on the book's password-protected Companion Website—see the access card information on the inside front cover.
- ***Debugger appendices.*** On the book's Companion Website we provide Appendix H, Using the Visual Studio Debugger, Appendix I, Using the GNU C++ Debugger and Appendix J, Using the Xcode Debugger.

New in This Edition

- Discussions of the new C++14 capabilities.
- Further integration of C++11 capabilities into the code examples, because the latest compilers are now supporting these features.
- Uniform initialization with list initializer syntax.
- Always using braces in control statements, even for single-statement bodies:

```
if (condition) {
   single-statement or multi-statement body
}
```

- Replaced the `Gradebook` class with `Account`, `Student` and `DollarAmount` class case studies in Chapters 3, 4 and 5, respectively. `DollarAmount` processes monetary amounts precisely for business applications.
- C++14 digit separators in large numeric literals.
- *Type* `&x` is now *Type*`& x` in accordance with industry idiom.
- *Type* `*x` is now *Type*`* x` in accordance with industry idiom.
- Using C++11 scoped `enums` rather than traditional C `enums`.
- We brought our terminology in line with the C++ standard.
- Key terms in summaries now appear in bold for easy reference.
- Removed extra spaces inside `[]`, `()`, `<>` and `{}` delimiters.
- Replaced most `print` member functions with `toString` member functions to make classes more flexible—for example, returning a `string` gives the client code the option of displaying it on the screen, writing it to a file, concatenating it with other `strings`, etc.

- Now using `ostringstream` to create formatted `string`s for items like the `string` representations of a `Time`, rather than outputting formatted data directly to the standard output.
- For simplicity, we deferred using the three-file architecture from Chapter 3 to Chapter 9, so all early class examples define the entire class in a header.
- We reimplement Chapter 10's `Array` class operator-overloading example with `unique_ptr`s in Chapter 24. Using raw pointers and dynamic-memory allocation with `new` and `delete` is a source of subtle programming errors, especially "memory leaks"—`unique_ptr` and the other smart pointer types help prevent such errors.
- Using lambdas rather than function pointers in Chapter 16, Standard Library Algorithms. This will get readers comfortable with lambdas, which can be combined with various Standard Library algorithms to perform functional programming in C++. We're planning a more in-depth treatment of functional programming for *C++ How to Program, 11/e*.
- Enhanced Chapter 24 with additional C++14 features.

Object-Oriented Programming

- ***Early-objects approach.*** The book introduces the basic concepts and terminology of object technology in Chapter 1. You'll develop your first customized classes and objects in Chapter 3. We worked hard to make this chapter especially accessible to novices. Presenting objects and classes early gets you "thinking about objects" immediately and mastering these concepts more thoroughly.[2]
- ***C++ Standard Library `string`.*** C++ offers *two* types of strings—`string` class objects (which we begin using in Chapter 3) and C-style pointer-based strings. We've replaced most occurrences of C strings with instances of C++ class `string` to make programs more robust and eliminate many of the security problems of C strings. We continue to discuss C strings later in the book to prepare you for working with the legacy code in industry. In new development, you should favor `string` objects.
- ***C++ Standard Library `array`.*** C++ offers *three* types of arrays—`array`s and `vector`s (which we start using in Chapter 7) and C-style, pointer-based arrays which we discuss in Chapter 8. Our primary treatment of arrays uses the Standard Library's `array` and `vector` class templates instead of built-in, C-style, pointer-based arrays. We still cover built-in arrays because they remain useful in C++ and so that you'll be able to read legacy code. In new development, you should favor class template `array` and `vector` objects.
- ***Crafting valuable classes.*** A key goal of this book is to prepare you to build valuable reusable classes. Chapter 10 begins with a test-drive of class template `string` so you can see an elegant use of operator overloading before you implement your own customized class with overloaded operators. In the Chapter 10 case study,

2. For courses that require a late-objects approach, consider our pre-C++11 book *C++ How to Program, Late Objects Version*, which begins with six chapters on programming fundamentals (including two on control statements) and continues with seven chapters that gradually introduce object-oriented programming concepts.

you'll build your own custom `Array` class, then in the Chapter 18 exercises you'll convert it to a class template. You will have truly crafted valuable classes.

- ***Case studies in object-oriented programming.*** We provide several well-engineered real-world case studies, including the `Account` class in Chapter 3, `Student` class in Chapter 4, `DollarAmount` class in Chapter 5, `GradeBook` class in Chapter 7, the `Time` class in Chapter 9, the `Employee` class in Chapters 11–12 and more.
- ***Optional case study: Using the UML to develop an object-oriented design and C++ implementation of an ATM.*** The UML™ (Unified Modeling Language™) is the industry-standard graphical language for modeling object-oriented systems. We introduce the UML in the early chapters. Online Chapters 25 and 26 include an *optional* object-oriented design case study using the UML. We design and fully implement the software for a simple automated teller machine (ATM). We analyze a typical requirements document that specifies the system to be built. We determine the classes needed to implement that system, the attributes the classes need to have, the behaviors the classes need to exhibit and we specify how objects of the classes must interact with one another to meet the system requirements. From the design we produce a complete C++ implementation. Students often report that the case study helps them "tie it all together" and truly understand object orientation.
- ***Understanding how polymorphism works.*** Chapter 12 contains a detailed diagram and explanation of how C++ typically implements polymorphism, `virtual` functions and dynamic binding "under the hood."
- ***Object-oriented exception handling.*** We integrate basic exception handling early in the book (Chapter 7). Instructors can easily pull more detailed material forward from Chapter 17, Exception Handling: A Deeper Look.
- ***Custom template-based data structures.*** We provide a rich multi-chapter treatment of data structures—see the Data Structures module in the chapter dependency chart (Fig. 5).
- ***Three programming paradigms.*** We discuss *structured programming*, *object-oriented programming* and *generic programming*.

Hundreds of Code Examples

We include a broad range of example programs selected from computer science, information technology, business, simulation, game playing and other topics. The examples are accessible to students in novice-level and intermediate-level C++ courses (Fig. 2).

Examples	
`Account` class	`BasePlusCommissionEmployee` class
`Array` class case study	Binary tree creation and traversal
`Author` class	`BinarySearch` test program
Bank account program	Card shuffling and dealing
Bar chart printing program	`ClientData` class

Fig. 2 | A sampling of the book's examples. (Part 1 of 2.)

Examples	
`CommissionEmployee` class	Poll analysis program
Comparing `strings`	Polymorphism demonstration
Compilation and linking process	Preincrementing and postincrementing
Compound interest calculations with `for`	`priority_queue` adapter class
Converting `string` objects to C strings	`queue` adapter class
Counter-controlled repetition	Random-access files
Dice game simulation	Random number generation
`DollarAmount` class	Recursive function `factorial`
Credit inquiry program	Rolling a six-sided die 60,000,000 times
`Date` class	`SalariedEmployee` class
Downcasting and runtime type information	`SalesPerson` class
`Employee` class	Searching and sorting algorithms of the Standard Library
`explicit` constructor	Sequential files
`fibonacci` function	`set` class template
`fill` algorithms	`shared_ptr` program
Specializations of function template `printArray`	`stack` adapter class
`generate` algorithms	`Stack` class
`GradeBook` Class	Stack unwinding
Initializing an array in a declaration	Standard Library `string` class program
Input from an `istringstream` object	Stream manipulator `showbase`
Iterative factorial solution	`string` assignment and concatenation
Lambda expressions	`string` member function `substr`
Linked list manipulation	`Student` class
`map` class template	Summing integers with the `for` statement
Mathematical algorithms of the Standard Library	`Time` class
`maximum` function template	`unique_ptr` object managing dynamically allocated memory
Merge sort program	Validating user input with regular expressions
`multiset` class template	`vector` class template
`new` throwing `bad_alloc` on failure	
`PhoneNumber` class	

Fig. 2 | A sampling of the book's examples. (Part 2 of 2.)

Exercises

- ***Self-Review Exercises and Answers.*** Extensive self-review exercises *and* answers are included for self-study.
- ***Interesting, entertaining and challenging exercises.*** Each chapter concludes with a substantial set of exercises, including simple recall of important terminology and concepts, identifying the errors in code samples, writing individual program statements, writing small portions of C++ classes and member and non-member functions, writing complete programs and implementing major projects. Figure 3 lists a sampling of the book's exercises, including our *Making a Difference* exercises, which encourage you to use computers and the Internet to research and work on significant social problems. We hope you'll approach these exercises with your own values, politics and beliefs.

Exercises		
Airline Reservations System	Credit Limits	Phishing Scanner
Advanced String-Manipulation	Crossword Puzzle Generator	Pig Latin
Bubble Sort	Cryptograms	Polymorphic Banking Program Using Account Hierarchy
Building Your Own Compiler	De Morgan's Laws	Pythagorean Triples
Building Your Own Computer	Dice Rolling	Salary Calculator
Calculating Salaries	Eight Queens	Sieve of Eratosthenes
`CarbonFootprint` Abstract Class: Polymorphism	Emergency Response	Simple Decryption
Card Shuffling and Dealing	Enforcing Privacy with Cryptography	Simple Encryption
Computer-Assisted Instruction	Facebook User Base Growth	SMS Language
Computer-Assisted Instruction: Difficulty Levels	Fibonacci Series	Spam Scanner
Computer-Assisted Instruction: Monitoring Student Performance	Gas Mileage	Spelling Checker
Computer-Assisted Instruction: Reducing Student Fatigue	Global Warming Facts Quiz	Target-Heart-Rate Calculator
Computer-Assisted Instruction: Varying the Types of Problems	Guess the Number Game	Tax Plan Alternatives; The "Fair Tax"
Cooking with Healthier Ingredients	Hangman Game	Telephone number word generator
Craps Game Modification	Health Records	"The Twelve Days of Christmas" Song
	Knight's Tour	Tortoise and the Hare Simulation
	Limericks	Towers of Hanoi
	Maze Traversal: Generating Mazes Randomly	World Population Growth
	Morse Code	
	Payroll System Modification	
	Peter Minuit Problem	

Fig. 3 | A sampling of the book's exercises.

Illustrations and Figures

Abundant tables, line drawings, UML diagrams, programs and program outputs are included. A sampling of the book's drawings and diagrams is shown in (Fig. 4).

Drawings and diagrams		
Main text drawings and diagrams		
`Account` class diagrams	`while` repetition statement UML activity diagram	Pass-by-value and pass-by-reference analysis
Data hierarchy	`for` repetition statement UML activity diagram	Inheritance hierarchy diagrams
Multiple-source-file compilation and linking	`do...while` repetition statement UML activity diagram	Function-call stack and activation records
Order in which a second-degree polynomial is evaluated	`switch` multiple-selection statement activity diagram	Recursive calls to function `fibonacci`
`if` single-selection statement activity diagram	C++'s single-entry/single-exit control statements	Pointer arithmetic diagrams
`if...else` double-selection statement activity diagram		`CommunityMember` Inheritance hierarchy

Fig. 4 | A sampling of the book's drawings and diagrams. (Part 1 of 2.)

Drawings and diagrams

Main text drawings and diagrams (cont.)

- `Shape` inheritance hierarchy
- `public`, `protected` and `private` inheritance
- `Employee` hierarchy UML class diagram
- How `virtual` function calls work
- Two self-referential class objects linked together
- Graphical representation of a list
- Operation `insertAtFront` represented graphically
- Operation `insertAtBack` represented graphically
- Operation `removeFromFront` represented graphically
- Operation `removeFromBack` represented graphically
- Circular, singly linked list
- Doubly linked list
- Circular, doubly linked list
- Graphical representation of a binary tree

(Optional) ATM Case Study drawings and diagrams

- Use case diagram for the ATM system from the User's perspective
- Class diagram showing an association among classes
- Class diagram showing composition relationships
- Class diagram for the ATM system model
- Classes with attributes
- State diagram for the ATM
- Activity diagram for a `BalanceInquiry` transaction
- Activity diagram for a `Withdrawal` transaction
- Classes in the ATM system with attributes and operations
- Communication diagram of the ATM executing a balance inquiry
- Communication diagram for executing a balance inquiry
- Sequence diagram that models a `Withdrawal` executing
- Use case diagram for a modified version of our ATM system that also allows users to transfer money between accounts
- Class diagram showing composition relationships of a class `Car`
- Class diagram for the ATM system model including class `Deposit`
- Activity diagram for a `Deposit` transaction
- Sequence diagram that models a `Deposit` executing

Fig. 4 | A sampling of the book's drawings and diagrams. (Part 2 of 2.)

Dependency Chart

C++ How to Program, 10/e is appropriate for most introductory one-and-two-course programming sequences, often called CS1 and CS2. The chart in Fig. 5 shows the dependencies among the chapters to help instructors plan their syllabi. The chart shows the book's modular organization.

Teaching Approach

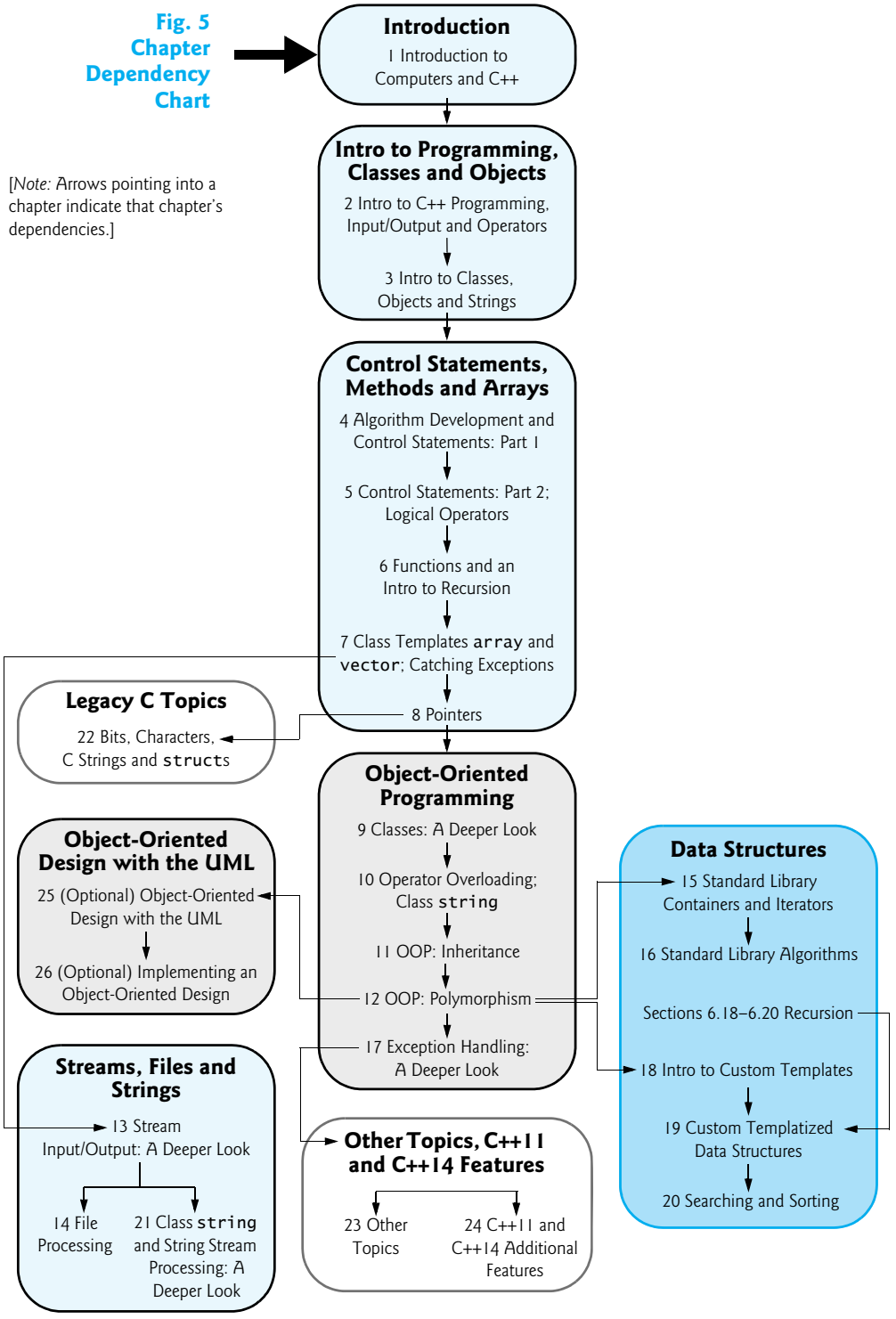
C++ How to Program, 10/e, contains a rich collection of examples. We stress program clarity and concentrate on building well-engineered software.

Live-code approach. The book is loaded with "live-code" examples—most new concepts are presented in *complete working C++ applications*, followed by one or more executions showing program inputs and outputs.

Rich early coverage of C++ fundamentals. Chapter 2 provides a friendly introduction to C++ programming. We include in Chapters 4 and 5 a clear treatment of control statements and algorithm development.

Fig. 5 Chapter Dependency Chart

[*Note:* Arrows pointing into a chapter indicate that chapter's dependencies.]



Syntax coloring. For readability, we syntax color all the C++ code, similar to the way most C++ integrated-development environments and code editors syntax color code. Our coloring conventions are as follows:

```
comments appear like this
keywords appear like this
constants and literal values appear like this
all other code appears in black
```

Code highlighting. We place shaded rectangles around the new features in each program.

Using fonts for emphasis. We color the defining occurrence of each key term in **bold colored** text for easy reference. We emphasize on-screen components in the **bold Helvetica** font (e.g., the **File** menu) and C++ program text in the `Lucida` font (for example, `int x = 5;`).

Objectives. We clearly state the chapter objectives.

Programming tips. We include programming tips to help you focus on key aspects of program development. These tips and practices represent the best we've gleaned from a combined eight decades of teaching and industry experience.

Good Programming Practices

The Good Programming Practices *call attention to techniques that will help you produce programs that are clearer, more understandable and more maintainable.*

Common Programming Errors

Pointing out these Common Programming Errors *reduces the likelihood that you'll make them.*

Error-Prevention Tips

These tips contain suggestions for exposing and removing bugs from your programs; many describe aspects of C++ that prevent bugs from getting into programs in the first place.

Performance Tips

These tips highlight opportunities for making your programs run faster or minimizing the amount of memory that they occupy.

Portability Tips

These tips help you write code that will run on a variety of platforms.

Software Engineering Observations

These tips highlight architectural and design issues that affect the construction of software systems, especially large-scale systems.

Summary Bullets. We present a section-by-section, bullet-list summary of each chapter. Each key term is in **bold** followed by the page number of the term's defining occurrence.

Index. For convenient reference, we've included an extensive index, with defining occurrences of key terms highlighted with a **bold** page number.

Secure C++ Programming

It's difficult to build industrial-strength systems that stand up to attacks from viruses, worms, and other forms of "malware." Today, via the Internet, such attacks can be instantaneous and global in scope. Building security into software from the beginning of the development cycle can greatly reduce vulnerabilities.

The CERT® Coordination Center (`www.cert.org`) was created to analyze and respond promptly to attacks. CERT—the Computer Emergency Response Team—is a government-funded organization within the Carnegie Mellon University Software Engineering Institute™. CERT publishes and promotes secure coding standards for various popular programming languages to help software developers implement industrial-strength systems which avoid the programming practices that leave systems open to attacks.

We'd like to thank Robert C. Seacord, an adjunct professor in the Carnegie Mellon University School of Computer Science and former Secure Coding Manager at CERT. Mr. Seacord was a technical reviewer for our book, *C How to Program, 7/e*, where he scrutinized our C programs from a security standpoint, recommending that we adhere to key guidelines of the *CERT C Secure Coding Standard*.

We've done the same for *C++ How to Program, 10/e*, adhering to key guidelines of the *CERT C++ Secure Coding Standard*, which you can find at:

```
http://www.securecoding.cert.org
```

We were pleased to discover that we've already been recommending many of these coding practices in our books since the early 1990s. We upgraded our code and discussions to conform to these practices, as appropriate for an introductory/intermediate-level textbook. If you'll be building industrial-strength C++ systems, consider reading *Secure Coding in C and C++, Second Edition* (Robert Seacord, Addison-Wesley Professional, 2013).

Online Chapters, Appendices and Other Content

The book's Companion Website, which is accessible at

```
http://www.pearsonglobaleditions.com/deitel
```

(see the inside front cover for your access key) contains the following videos as well as chapters and appendices in searchable PDF format:

- *VideoNotes*—The Companion Website (see the inside front cover for your access key) also includes extensive videos. Watch and listen as co-author Paul Deitel discusses in-depth the key code examples from the book's core programming-fundamentals and object-oriented-programming chapters.
- Chapter 23, Other Topics
- Chapter 24, C++11 and C++14 Additional Topics
- Chapter 25, ATM Case Study, Part 1: Object-Oriented Design with the UML
- Chapter 26, ATM Case Study, Part 2: Implementing an Object-Oriented Design
- Appendix F, C Legacy Code Topics
- Appendix G, UML 2: Additional Diagram Types

- Appendix H, Using the Visual Studio Debugger
- Appendix I, Using the GNU C++ Debugger
- Appendix J, Using the Xcode Debugger
- Building Your Own Compiler exercise descriptions from Chapter 19 (posted at the Companion Website.)

Obtaining the Software Used in *C++ How to Program, 10/e*

We wrote the code examples in *C++ How to Program, 10/e* using the following free C++ development tools:

- Microsoft's free Visual Studio Community 2015 edition, which includes Visual C++ and other Microsoft development tools. This runs on Windows and is available for download at

```
https://www.visualstudio.com/products/visual-studio-community-vs
```

- GNU's free GNU C++ 5.2.1. GNU C++ is already installed on most Linux systems and can also be installed on Mac OS X and Windows systems. There are many versions of Linux—known as Linux distributions—that use different techniques for performing software upgrades. Check your distribution's online documentation for information on how to upgrade GNU C++ to the latest version. GNU C++ is available at

```
http://gcc.gnu.org/install/binaries.html
```

- Apple's free Xcode, which OS X users can download from the Mac App Store—click the app's icon in the dock at the bottom of your screen, then search for Xcode in the app store.

Instructor Supplements

The following supplements are available to *qualified instructors only* through Pearson Education's Instructor Resource Center (`http://www.pearsonglobaleditions.com/deitel`):

- ***Solutions Manual*** contains solutions to most of the end-of-chapter exercises. We include *Making a Difference* exercises, many with solutions. **Please do not write to us requesting access to the Pearson Instructor's Resource Center. Access is restricted to college instructors teaching from the book.** Instructors may obtain access only through their Pearson representatives.

  Solutions are *not* provided for "project" exercises. Check out our Programming Projects Resource Center for lots of additional exercise and project possibilities.

```
http://www.deitel.com/ProgrammingProjects
```

- ***Test Item File*** of multiple-choice questions.
- ***Customizable PowerPoint® slides*** containing all the code and figures in the text, plus bulleted items that summarize key points in the text.

Online Practice and Assessment with MyProgrammingLab™

MyProgrammingLab™ helps students fully grasp the logic, semantics, and syntax of programming. Through practice exercises and immediate, personalized feedback, MyProgrammingLab improves the programming competence of beginning students who often struggle with the basic concepts and paradigms of popular high-level programming languages.

An optional self-study and homework tool, a MyProgrammingLab course consists of hundreds of small practice problems organized around the structure of this textbook. For students, the system automatically detects errors in the logic and syntax of their code submissions and offers targeted hints that enable students to figure out what went wrong—and why. For instructors, a comprehensive gradebook tracks correct and incorrect answers and stores the code inputted by students for review.

For a full demonstration, to see feedback from instructors and students or to get started using MyProgrammingLab in your course, visit

```
http://www.myprogramminglab.com
```

Acknowledgments

We'd like to thank Barbara Deitel of Deitel & Associates, Inc. for long hours devoted to this project. She painstakingly researched the new capabilities of C++11 and C++14.

We're fortunate to have worked with the dedicated team of publishing professionals at Pearson Higher Education. We appreciate the guidance, wisdom and energy of Tracy Johnson, Executive Editor, Computer Science. Kristy Alaura did an extraordinary job recruiting the book's reviewers and managing the review process. Bob Engelhardt did a wonderful job bringing the book to publication.

Finally, thanks to Abbey Deitel, former President of Deitel & Associates, Inc., and a graduate of Carnegie Mellon University's Tepper School of Management where she received a B.S. in Industrial Management. Abbey managed the business operations of Deitel & Associates, Inc. for 17 years, along the way co-authoring a number of our publications, including the previous *C++ How to Program* editions' versions of Chapter 1.

Reviewers

We wish to acknowledge the efforts of our reviewers. Over its ten editions, the book has been scrutinized by academics teaching C++ courses, current and former members of the C++ standards committee and industry experts using C++ to build industrial-strength, high-performance systems. They provided countless suggestions for improving the presentation. Any remaining flaws in the book are our own.

Tenth Edition reviewers: Chris Aburime, Minnesota State Colleges and Universities System; Gašper Ažman, A9.com Search Technologies and Co-Author of *C++ Today: The Beast is Back*; Danny Kalev, Intel and Former Member of the C++ Standards Committee; Renato Golin, LLVM Tech Lead at Linaro and Co-Owner for the ARM Target in LLVM; Gordon Hogenson, Microsoft, Author of *Foundations of C++/CLI: The Visual C++ Language for .NET 3*; Jonathan Wakely, Redhat, ISO C++ Committee Secretary; José Antonio González Seco, Parliament of Andalusia; Dean Michael Berris, Google, Maintainer of cpp-netlib and Former ISO C++ Committee Member.

Ninth Edition post-publication academic reviewers: Stefano Basagni, Northeastern University; Amr Elkady, Diablo Valley College; Chris Aburime, Minnesota State Colleges and Universities System.

Other recent edition reviewers: Virginia Bailey (Jackson State University), Ed James-Beckham (Borland), Thomas J. Borrelli (Rochester Institute of Technology), Ed Brey (Kohler Co.), Chris Cox (Adobe Systems), Gregory Dai (eBay), Peter J. DePasquale (The College of New Jersey), John Dibling (SpryWare), Susan Gauch (University of Arkansas), Doug Gregor (Apple, Inc.), Jack Hagemeister (Washington State University), Williams M. Higdon (University of Indiana), Anne B. Horton (Lockheed Martin), Terrell Hull (Logicalis Integration Solutions), Linda M. Krause (Elmhurst College), Wing-Ning Li (University of Arkansas), Dean Mathias (Utah State University), Robert A. McLain (Tidewater Community College), James P. McNellis (Microsoft Corporation), Robert Myers (Florida State University), Gavin Osborne (Saskatchewan Institute of Applied Science and Technology), Amar Raheja (California State Polytechnic University, Pomona), April Reagan (Microsoft), Robert C. Seacord (Secure Coding Manager at SEI/CERT, author of *Secure Coding in C and C++*), Raymond Stephenson (Microsoft), Dave Topham (Ohlone College), Anthony Williams (author and C++ Standards Committee member) and Chad Willwerth (University Washington, Tacoma).

As you read the book, we'd sincerely appreciate your comments, criticisms and suggestions for improving the text. Please address all correspondence to:

```
deitel@deitel.com
```

We'll respond promptly. We enjoyed writing *C++ How to Program, Tenth Edition*. We hope you enjoy reading it!

Paul Deitel
Harvey Deitel

Acknowledgments for the Global Edition

Pearson would like to thank and acknowledge the following people for their contributions to the Global Edition.

Contributors
Rosanne Els, University of Kawazulu Natal; Almasi S. Maguya, Mzumbe University; Khyat Sharma.

Reviewers
T.V. Gopal, Anna University; Subrajeet Mohapatra, Birla Institute of Technology; Prabhat Verma, Harcourt Butler Technological Institute.

About the Authors

Paul Deitel, CEO and Chief Technical Officer of Deitel & Associates, Inc., has over 30 years of experience in computing. He is a graduate of MIT, where he studied Information Technology. He holds the Java Certified Programmer and Java Certified Developer designations and is an Oracle Java Champion. Paul was also named as a Microsoft® Most Valuable Professional (MVP) for C# in 2012–2014. Through Deitel & Associates, Inc., he has delivered hundreds of programming courses worldwide to clients, including Cisco, IBM, Siemens, Sun Microsystems, Dell, Fidelity, NASA at the Kennedy Space Center, the National Severe Storm Laboratory, White Sands Missile Range, Rogue Wave Software, Boeing, SunGard, Nortel Networks, Puma, iRobot, Invensys and many more. He and his co-author, Dr. Harvey Deitel, are the world's best-selling programming-language textbook/professional book/video authors.

Dr. Harvey Deitel, Chairman and Chief Strategy Officer of Deitel & Associates, Inc., has over 50 years of experience in the computer field. Dr. Deitel earned B.S. and M.S. degrees in Electrical Engineering from MIT and a Ph.D. in Mathematics from Boston University—he studied computing in each of these programs before they spun off Computer Science programs. He has extensive college teaching experience, including earning tenure and serving as the Chairman of the Computer Science Department at Boston College before founding Deitel & Associates, Inc., in 1991 with his son, Paul. The Deitels' publications have earned international recognition, with translations published in Japanese, German, Russian, Spanish, French, Polish, Italian, Simplified Chinese, Traditional Chinese, Korean, Portuguese, Greek, Urdu and Turkish. Dr. Deitel has delivered hundreds of programming courses to academic, corporate, government and military clients.

About Deitel & Associates, Inc.

Deitel & Associates, Inc., founded by Paul Deitel and Harvey Deitel, is an internationally recognized authoring and corporate training organization, specializing in computer programming languages, object technology, Internet and web software technology, and Android and iOS app development. The company's clients include academic institutions, many of the world's largest corporations, government agencies and branches of the military. The company offers instructor-led training courses delivered at client sites worldwide on major programming languages and platforms, including C++, C, Java™, Android app development, iOS app development, Swift™, Visual C#®, Visual Basic®, Internet and web programming and a growing list of additional programming and software-development courses.

Before You Begin

This section contains information you should review before using this book and instructions to ensure that your computer is set up properly to compile the example programs.

Font and Naming Conventions

We use fonts to distinguish between features, such as menu names, menu items, and other elements that appear in your IDE (Integrated Development Environment), such as Microsoft's Visual Studio. Our convention is to emphasize IDE features in a sans-serif bold **Helvetica** font (for example, **File** menu) and to emphasize program text in a sans-serif `Lucida` font (for example, `bool x = true;`).

Obtaining the Software Used in *C++ How to Program, 10/e*

Before reading this book, you should download and install a C++ compiler. We wrote *C++ How to Program, 10/e*'s code examples using the following free C++ development tools:

- Microsoft's free Visual Studio Community 2015 edition, which includes the Visual C++ compiler and other Microsoft development tools. This runs on Windows and is available for download at

```
https://www.visualstudio.com/products/visual-studio-community-vs
```

- GNU's free GNU C++ 5.2.1 compiler. GNU C++ is already installed on most Linux systems and also can be installed on Mac OS X and Windows systems. There are many versions of Linux, known as Linux distributions, that use different techniques for performing software upgrades. Check your distribution's online documentation for information on how to upgrade GNU C++ to the latest version. GNU C++ is available at

```
http://gcc.gnu.org/install/binaries.html
```

- Apple's free Xcode, which OS X users can download from the Mac App Store—click the app's icon in the dock at the bottom of your Mac screen, then search for Xcode in the app store.

We also provide links to our getting-started videos for each of these C++ tools at:

```
http://www.deitel.com/books/cpphtp10
```

Obtaining the Code Examples

The examples for *C++ How to Program, 10/e* are available for download at

```
http://www.pearsonglobaleditions.com/deitel
```

Click the **Download Code Examples** link to download the ZIP archive file to your computer. Write down the location where you saved the file—most browsers will save the file into your user account's `Downloads` folder.

Throughout the book, steps that require you to access our example code on your computer assume that you've extracted the examples from the ZIP file and placed them in `C:\examples` on Windows or in your user account's `Documents` directory on other platforms. You can extract them anywhere you like, but if you choose a different location, you'll need to update our steps accordingly.

Creating Projects

In Section 1.10, we demonstrate how to compile and run programs with

- Microsoft Visual Studio Community 2015 edition on Windows (Section 1.10.1)
- GNU C++ 5.2.1 on Linux (Section 1.10.2)
- Apple Xcode on OS X (Section 1.10.3)

For GNU C++ and Xcode, you must compile your programs with C++14. To do so in GNU C++, include the option `-std=c++14` when you compile your code, as in:

```
g++ -std=c++14 YourFileName.cpp
```

For Xcode, after following Section 1.10.3's steps to create a project:

1. Select the root node at the top of the Xcode **Project** navigator.
2. Click the **Build Settings** tab in the **Editors** area.
3. Scroll down to the **Apple LLVM 7.0 - Language - C++** section.
4. For the **C++ Language Dialect** option, select **C++14 [–std=c++14]**.

Getting Your C++ Questions Answered

As you read the book, if you have questions, we're easy to reach at

```
deitel@deitel.com
```

We'll respond promptly.

In addition, the web is loaded with programming information. An invaluable resource for nonprogrammers and programmers alike is the website

```
http://stackoverflow.com
```

on which you can:

- Search for answers to most common programming questions
- Search for error messages to see what causes them
- Ask programming questions to get answers from programmers worldwide
- Gain valuable insights about programming in general

Online C++ Documentation

For documentation on the C++ Standard Library, visit

```
http://cppreference.com
```

and be sure to check out the C++ FAQ at

```
https://isocpp.org/faq
```

Introduction to Computers and C++

1

Objectives

In this chapter you'll learn:

- Exciting recent developments in the computer field.
- Computer hardware, software and networking basics.
- The data hierarchy.
- The different types of programming languages.
- Basic object-technology concepts.
- Some basics of the Internet and the World Wide Web.
- A typical C++ program development environment.
- To test-drive a C++ application.
- Some key recent software technologies.
- How computers can help you make a difference.

Outline

1.1 Introduction

Welcome to C++—a powerful computer programming language that's appropriate for technically oriented people with little or no programming experience, and for experienced programmers to use in building substantial information systems. You're already familiar with the powerful tasks computers perform. Using this textbook, you'll write instructions commanding computers to perform those kinds of tasks. *Software* (i.e., the instructions you write) controls *hardware* (i.e., computers).

You'll learn *object-oriented programming*—today's key programming methodology. You'll create many *software objects* that model *things* in the real world.

C++ is one of today's most popular software development languages. This text provides an introduction to programming in C++11 and C++14—the latest versions standardized through the International Organization for Standardization (ISO) and the International Electrotechnical Commission (IEC).

11
14

As of 2008 there were more than a billion general-purpose computers in use. Today, various websites say that number is approximately two billion, and according to the real-time tracker at `gsmaintelligence.com`, there are now more mobile devices than there are people in the world. According to the International Data Corporation (IDC), the number of mobile Internet users will top two billion in 2016.[1] Smartphone sales surpassed personal computer sales in 2011.[2] Tablet sales were expected to overtake personal-computer sales

1. `https://www.idc.com/getdoc.jsp?containerId=prUS40855515`.
2. `http://www.mashable.com/2012/02/03/smartphone-sales-overtake-pcs/`.

by 2015.[3] By 2017, the smartphone/tablet app market is expected to exceed $77 billion.[4] This explosive growth is creating significant opportunities for programming mobile applications.

1.2 Computers and the Internet in Industry and Research

These are exciting times in the computer field. Many of the most influential and successful businesses of the last two decades are technology companies, including Apple, IBM, Hewlett Packard, Dell, Intel, Motorola, Cisco, Microsoft, Google, Amazon, Facebook, Twitter, eBay and many more. These companies are major employers of people who study computer science, computer engineering, information systems or related disciplines. At the time of this writing, Apple was the most valuable company in the world. Figure 1.1 provides a few examples of the ways in which computers are improving people's lives in research, industry and society.

Name	Description
Electronic health records	These might include a patient's medical history, prescriptions, immunizations, lab results, allergies, insurance information and more. Making this information available to health care providers across a secure network improves patient care, reduces the probability of error and increases overall efficiency of the health-care system, helping control costs.
Human Genome Project	The Human Genome Project was founded to identify and analyze the 20,000+ genes in human DNA. The project used computer programs to analyze complex genetic data, determine the sequences of the billions of chemical base pairs that make up human DNA and store the information in databases which have been made available over the Internet to researchers in many fields.
AMBER™ Alert	The AMBER (America's Missing: Broadcast Emergency Response) Alert System is used to find abducted children. Law enforcement notifies TV and radio broadcasters and state transportation officials, who then broadcast alerts on TV, radio, computerized highway signs, the Internet and wireless devices. AMBER Alert recently partnered with Facebook, whose users can "Like" AMBER Alert pages by location to receive alerts in their news feeds.
World Community Grid	People worldwide can donate their unused computer processing power by installing a free secure software program that allows the World Community Grid (`http://www.worldcommunitygrid.org`) to harness unused capacity. This computing power, accessed over the Internet, is used in place of expensive supercomputers to conduct scientific research projects that are making a difference—providing clean water to third-world countries, fighting cancer, growing more nutritious rice for regions fighting hunger and more.

Fig. 1.1 | A few uses for computers. (Part 1 of 3.)

3. `http://www.forbes.com/sites/louiscolumbus/2014/07/18/gartner-forecasts-tablet-shipments-will-overtake-pcs-in-2015/`.
4. `http://www.entrepreneur.com/article/236832`.

Name	Description
Cloud computing	**Cloud computing** allows you to use software, hardware and information stored in the "cloud"—i.e., accessed on remote computers via the Internet and available on demand—rather than having it stored on your personal computer. These services allow you to increase or decrease resources to meet your needs at any given time, so they can be more cost effective than purchasing expensive hardware to ensure that you have enough storage and processing power to meet your needs at their peak levels. Using cloud-computing services shifts the burden of managing these applications from the business to the service provider, saving businesses money.
Medical imaging	X-ray computed tomography (CT) scans, also called CAT (computerized axial tomography) scans, take X-rays of the body from hundreds of different angles. Computers are used to adjust the intensity of the X-rays, optimizing the scan for each type of tissue, then to combine all of the information to create a 3D image. MRI scanners use a technique called magnetic resonance imaging, also to produce internal images noninvasively.
GPS	Global Positioning System (GPS) devices use a network of satellites to retrieve location-based information. Multiple satellites send time-stamped signals to the GPS device, which calculates the distance to each satellite, based on the time the signal left the satellite and the time the signal arrived. This information helps determine the device's exact location. GPS devices can provide step-by-step directions and help you locate nearby businesses (restaurants, gas stations, etc.) and points of interest. GPS is used in numerous location-based Internet services such as check-in apps to help you find your friends (e.g., Foursquare and Facebook), exercise apps such as RunKeeper that track the time, distance and average speed of your outdoor jog, dating apps that help you find a match nearby and apps that dynamically update changing traffic conditions.
Robots	Robots can be used for day-to-day tasks (e.g., iRobot's Roomba vacuuming robot), entertainment (e.g., robotic pets), military combat, deep sea and space exploration (e.g., NASA's Mars rover Curiosity) and more. RoboEarth (`www.roboearth.org`) is "a World Wide Web for robots." It allows robots to learn from each other by sharing information and thus improving their abilities to perform tasks, navigate, recognize objects and more.
E-mail, Instant Messaging, Video Chat and FTP	Internet-based servers support all of your online messaging. E-mail messages go through a mail server that also stores the messages. Instant Messaging (IM) and Video Chat apps, such as Facebook Messenger, AIM, Skype, Yahoo! Messenger, Google Hangouts, Trillian and others allow you to communicate with others in real time by sending your messages and live video through servers. FTP (file transfer protocol) allows you to exchange files between multiple computers (for example, a client computer such as your desktop and a file server) over the Internet.
Internet TV	Internet TV set-top boxes (such as Apple TV, Android TV, Roku and TiVo) allow you to access an enormous amount of content on demand, such as games, news, movies, television shows and more, and they help ensure that the content is streamed to your TV smoothly.

Fig. 1.1 | A few uses for computers. (Part 2 of 3.)

Name	Description
Streaming music services	Streaming music services (such as Apple Music, Pandora, Spotify, Last.fm and more) allow you listen to large catalogues of music over the web, create customized "radio stations" and discover new music based on your feedback.
Game programming	Global video-game revenues are expected to reach $107 billion by 2017 (`http://www.polygon.com/2015/4/22/8471789/worldwide-video-games-market-value-2015`). The most sophisticated games can cost over $100 million to develop, with the most expensive costing half a billion dollars (`http://www.gamespot.com/gallery/20-of-the-most-expensive-games-ever-made/2900-104/`). Bethesda's *Fallout 4* earned $750 million in its first day of sales (`http://fortune.com/2015/11/16/fallout4-is-quiet-best-seller/`)!

Fig. 1.1 | A few uses for computers. (Part 3 of 3.)

1.3 Hardware and Software

Computers can perform calculations and make logical decisions phenomenally faster than human beings can. Many of today's personal computers can perform billions of calculations in one second—more than a human can perform in a lifetime. *Supercomputers* are already performing *thousands of trillions (quadrillions)* of instructions per second! China's National University of Defense Technology's Tianhe-2 supercomputer can perform over 33 quadrillion calculations per second (33.86 *petaflops*)![5] To put that in perspective, *the Tianhe-2 supercomputer can perform in one second about 3 million calculations for every person on the planet!* And supercomputing "upper limits" are growing quickly.

Computers process data under the control of sequences of instructions called **computer programs**. These programs guide the computer through ordered actions specified by people called computer **programmers**. The programs that run on a computer are referred to as **software**. In this book, you'll learn a key programming methodology that's enhancing programmer productivity, thereby reducing software development costs—*object-oriented programming*.

A computer consists of various devices referred to as hardware (e.g., the keyboard, screen, mouse, hard disks, memory, DVD drives and processing units). Computing costs are *dropping dramatically*, owing to rapid developments in hardware and software technologies. Computers that might have filled large rooms and cost millions of dollars decades ago are now inscribed on silicon chips smaller than a fingernail, costing perhaps a few dollars each. Ironically, silicon is one of the most abundant materials on Earth—it's an ingredient in common sand. Silicon-chip technology has made computing so economical that computers have become a commodity.

1.3.1 Moore's Law

Every year, you probably expect to pay at least a little more for most products and services. The opposite has been the case in the computer and communications fields, especially with regard to the hardware supporting these technologies. For many decades, hardware costs have fallen rapidly.

5. `http://www.top500.org`.

Every year or two, the capacities of computers have approximately *doubled* inexpensively. This remarkable trend often is called **Moore's Law**, named for the person who identified it in the 1960s, Gordon Moore, co-founder of Intel—a leading manufacturer of the processors in today's computers and embedded systems. Moore's Law and related observations apply especially to the amount of memory that computers have for programs, the amount of secondary storage (such as disk storage) they have to hold programs and data over longer periods of time, and their processor speeds—the speeds at which they *execute* their programs (i.e., do their work). These increases make computers more capable, which puts greater demands on programming-language designers to innovate.

Similar growth has occurred in the communications field—costs have plummeted as enormous demand for communications *bandwidth* (i.e., information-carrying capacity) has attracted intense competition. We know of no other fields in which technology improves so quickly and costs fall so rapidly. Such phenomenal improvement is truly fostering the *Information Revolution*.

1.3.2 Computer Organization

Regardless of differences in *physical* appearance, computers can be envisioned as divided into various **logical units** or sections (Fig. 1.2).

Logical unit	Description
Input unit	This "receiving" section obtains information (data and computer programs) from **input devices** and places it at the disposal of the other units for processing. Most user input is entered into computers through keyboards, touch screens and mouse devices. Other forms of input include receiving voice commands, scanning images and barcodes, reading from secondary storage devices (such as hard drives, DVD drives, Blu-ray Disc™ drives and USB flash drives—also called "thumb drives" or "memory sticks"), receiving video from a webcam and having your computer receive information from the Internet (such as when you stream videos from YouTube® or download e-books from Amazon). Newer forms of input include position data from a GPS device, and motion and orientation information from an *accelerometer* (a device that responds to up/down, left/right and forward/backward acceleration) in a smartphone or game controller (such as Microsoft® Kinect® for Xbox®, Wii™ Remote and Sony® PlayStation® Move).
Output unit	This "shipping" section takes information the computer has processed and places it on various **output devices** to make it available for use outside the computer. Most information that's output from computers today is displayed on screens (including touch screens), printed on paper ("going green" discourages this), played as audio or video on PCs and media players (such as Apple's iPods) and giant screens in sports stadiums, transmitted over the Internet or used to control other devices, such as robots and "intelligent" appliances. Information is also commonly output to secondary storage devices, such as hard drives, DVD drives and USB flash drives. Popular recent forms of output are smartphone and game controller vibration, and virtual reality devices like Oculus Rift.

Fig. 1.2 | Logical units of a computer. (Part 1 of 2.)

Logical unit	Description
Memory unit	This rapid-access, relatively low-capacity "warehouse" section retains information that has been entered through the input unit, making it immediately available for processing when needed. The memory unit also retains processed information until it can be placed on output devices by the output unit. Information in the memory unit is *volatile*—it's typically lost when the computer's power is turned off. The memory unit is often called either **memory**, **primary memory** or **RAM** (Random Access Memory). Main memories on desktop and notebook computers contain as much as 128 GB of RAM, though 2 to 16 GB is most common. GB stands for gigabytes; a gigabyte is approximately one billion bytes. A **byte** is eight bits. A bit is either a `0` or a `1`.
Arithmetic and logic unit (ALU)	This "manufacturing" section performs *calculations*, such as addition, subtraction, multiplication and division. It also contains the *decision* mechanisms that allow the computer, for example, to compare two items from the memory unit to determine whether they're equal. In today's systems, the ALU is implemented as part of the next logical unit, the CPU.
Central processing unit (CPU)	This "administrative" section coordinates and supervises the operation of the other sections. The CPU tells the input unit when information should be read into the memory unit, tells the ALU when information from the memory unit should be used in calculations and tells the output unit when to send information from the memory unit to certain output devices. Many of today's computers have multiple CPUs and, hence, can perform many operations simultaneously. A **multi-core processor** implements multiple processors on a single integrated-circuit chip—a *dual-core processor* has two CPUs, a *quad-core processor* has four and an *octa-core processor* has eight. Today's desktop computers have processors that can execute billions of instructions per second. To take full advantage of multi-core architecture you need to write multithreaded applications, which we introduce in Section 24.3.
Secondary storage unit	This is the long-term, high-capacity "warehousing" section. Programs or data not actively being used by the other units normally are placed on secondary storage devices (e.g., your *hard drive*) until they're again needed, possibly hours, days, months or even years later. Information on secondary storage devices is *persistent*—it's preserved even when the computer's power is turned off. Secondary storage information takes much longer to access than information in primary memory, but its cost per unit is much less. Examples of secondary storage devices include hard drives, DVD drives and USB flash drives, some of which can hold over 2 TB (TB stands for terabytes; a terabyte is approximately one trillion bytes). Typical hard drives on desktop and notebook computers hold up to 2 TB, and some desktop hard drives can hold up to 6 TB.

Fig. 1.2 | Logical units of a computer. (Part 2 of 2.)

1.4 Data Hierarchy

Data items processed by computers form a **data hierarchy** that becomes larger and more complex in structure as we progress from the simplest data items (called "bits") to richer ones, such as characters and fields. Figure 1.3 illustrates a portion of the data hierarchy.

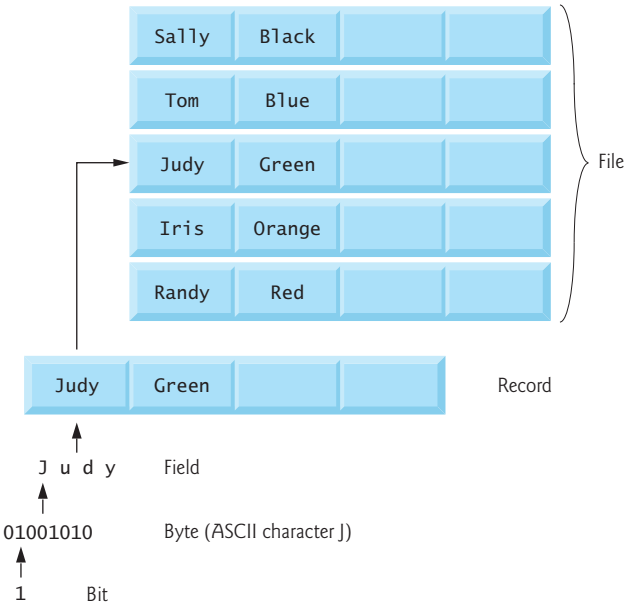


Fig. 1.3 | Data hierarchy.

Bits

The smallest data item in a computer can assume the value `0` or the value `1`. It's called a **bit** (short for "binary digit"—a digit that can assume one of *two* values). Remarkably, the impressive functions performed by computers involve only the simplest manipulations of `0`s and `1`s—*examining a bit's value*, *setting a bit's value* and *reversing a bit's value* (from `1` to `0` or from `0` to `1`).

Characters

It's tedious for people to work with data in the low-level form of bits. Instead, they prefer to work with *decimal digits* (0–9), *letters* (A–Z and a–z), and *special symbols* (e.g., $, @, %, &, *, (,), –, +, ", :, ? and /). Digits, letters and special symbols are known as **characters**. The computer's **character set** is the set of all the characters used to write programs and represent data items. Computers process only `1`s and `0`s, so a computer's character set represents every character as a pattern of `1`s and `0`s. C++ supports various character sets (including **Unicode**®), with some requiring more than one byte per character. Unicode supports many of the world's languages. See Appendix B for more information on the **ASCII (American Standard Code for Information Interchange)** character set—the popular subset of Unicode that represents uppercase and lowercase letters, digits and some common special characters.

Fields

Just as characters are composed of bits, **fields** are composed of characters or bytes. A field is a group of characters or bytes that conveys meaning. For example, a field consisting of uppercase and lowercase letters can be used to represent a person's name, and a field consisting of decimal digits could represent a person's age.

Records

Several related fields can be used to compose a **record**. In a payroll system, for example, the record for an employee might consist of the following fields (possible types for these fields are shown in parentheses):

- Employee identification number (a whole number)
- Name (a string of characters)
- Address (a string of characters)
- Hourly pay rate (a number with a decimal point)
- Year-to-date earnings (a number with a decimal point)
- Amount of taxes withheld (a number with a decimal point).

Thus, a record is a group of related fields. In the preceding example, all the fields belong to the *same* employee. A company might have many employees and a payroll record for each.

Files

A **file** is a group of related records. [*Note:* More generally, a file contains arbitrary data in arbitrary formats. In some operating systems, a file is viewed simply as a *sequence of bytes*—any organization of the bytes in a file, such as organizing the data into records, is a view created by the application programmer.] It's not unusual for an organization to have many files, some containing billions, or even trillions, of characters of information.

Database

A **database** is a collection of data organized for easy access and manipulation. The most popular model is the *relational database*, in which data is stored in simple *tables*. A table includes *records* and *fields*. For example, a table of students might include first name, last name, major, year, student ID number and grade-point-average fields. The data for each student is a record, and the individual pieces of information in each record are the fields. You can *search*, *sort* and otherwise manipulate the data based on its relationship to multiple tables or databases. For example, a university might use data from the student database in combination with data from databases of courses, on-campus housing, meal plans, etc.

Big Data

The amount of data being produced worldwide is enormous and growing quickly. According to IBM, approximately 2.5 quintillion bytes (2.5 *exabytes*) of data are created daily[6] and according to Salesforce.com, 90% of the world's data was created in just the past 12 months![7] According to an IDC study, the global data supply will reach 40 *zettabytes* (equal to 40 trillion gigabytes) annually by 2020.[8] Figure 1.4 shows some common byte measurements. **Big data** applications deal with massive amounts of data and this field is growing quickly, creating lots of opportunity for software developers. According to a study by Gartner Group, over 4 million IT jobs globally were expected to support big data in 2015.[9]

6. http://www-01.ibm.com/software/data/bigdata/what-is-big-data.html.
7. https://www.salesforce.com/blog/2015/10/salesforce-channel-ifttt.html.
8. http://recode.net/2014/01/10/stuffed-why-data-storage-is-hot-again-really/.
9. http://tech.fortune.cnn.com/2013/09/04/big-data-employment-boom/.

Unit	Bytes	Which is approximately
1 kilobyte (KB)	1024 bytes	10^3 (1024) bytes exactly
1 megabyte (MB)	1024 kilobytes	10^6 (1,000,000)bytes
1 gigabyte (GB)	1024 megabytes	10^9 (1,000,000,000) bytes
1 terabyte (TB)	1024 gigabytes	10^{12} (1,000,000,000,000) bytes
1 petabyte (PB)	1024 terabytes	10^{15} (1,000,000,000,000,000) bytes
1 exabyte (EB)	1024 petabytes	10^{18} (1,000,000,000,000,000,000) bytes
1 zettabyte (ZB)	1024 exabytes	10^{21} (1,000,000,000,000,000,000,000) bytes

Fig. 1.4 | Byte measurements.

1.5 Machine Languages, Assembly Languages and High-Level Languages

Programmers write instructions in various programming languages, some directly understandable by computers and others requiring intermediate *translation* steps.

Machine Languages

Any computer can directly understand only its own **machine language** (also called *machine code*), defined by its hardware architecture. Machine languages generally consist of numbers (ultimately reduced to 1s and 0s). Such languages are cumbersome for humans.

Assembly Languages

Programming in machine language was simply too slow and tedious for most programmers. Instead, they began using English-like *abbreviations* to represent elementary operations. These abbreviations formed the basis of **assembly languages**. *Translator programs* called **assemblers** were developed to convert assembly-language programs to machine language. Although assembly-language code is clearer to humans, it's incomprehensible to computers until translated to machine language.

High-Level Languages

To speed up the programming process further, **high-level languages** were developed in which single statements could be written to accomplish substantial tasks. High-level languages, such as C, C++, Java, C#, Swift and Visual Basic, allow you to write instructions that look more like everyday English and contain commonly used mathematical notations. Translator programs called **compilers** convert high-level language programs into machine language.

The process of compiling a large high-level language program into machine language can take a considerable amount of computer time. **Interpreter** programs were developed to execute high-level language programs directly (without the need for compilation), although more slowly than compiled programs. **Scripting languages** such as the popular web languages JavaScript and PHP are processed by interpreters.

Performance Tip 1.1

Interpreters have an advantage over compilers in Internet scripting. An interpreted program can begin executing as soon as it's downloaded to the client's machine, without needing to be compiled before it can execute. On the downside, interpreted scripts generally run slower and consume more memory than compiled code.

1.6 C and C++

C was implemented in 1972 by Dennis Ritchie at Bell Laboratories. It initially became widely known as the UNIX operating system's development language. Today, most of the code for general-purpose operating systems is written in C or C++.

C++ evolved from C, which is available for most computers and is hardware independent. With careful design, it's possible to write C programs that are **portable** to most computers.

The widespread use of C with various kinds of computers (sometimes called **hardware platforms**) unfortunately led to many variations. A standard version of C was needed. The American National Standards Institute (ANSI) cooperated with the International Organization for Standardization (ISO) to standardize C worldwide; the joint standard document was published in 1990.

C11 is the latest ANSI standard for the C programming language. It was developed to evolve the C language to keep pace with increasingly powerful hardware and ever more demanding user requirements. C11 also makes C more consistent with C++. For more information on C and C11, see our book *C How to Program, 8/e* and our C Resource Center (located at `http://www.deitel.com/C`).

C++, an extension of C, was developed by Bjarne Stroustrup in 1979 at Bell Laboratories. Originally called "C with Classes," it was renamed to C++ in the early 1980s. C++ provides a number of features that "spruce up" the C language, but more importantly, it provides capabilities for object-oriented programming that were inspired by the Simula simulation programming language. We say more about C++ and its current version in Section 1.14.

You'll begin developing customized, reusable classes and objects in Chapter 3. The book is object oriented, where appropriate, from the start and throughout the text.

We also provide an *optional* automated teller machine (ATM) case study in Chapters 25–26, which contains a complete C++ implementation. The case study presents a carefully paced introduction to object-oriented design using the UML—an industry standard graphical modeling language for developing object-oriented systems. We guide you through a friendly design and implementation experience intended for the novice.

C++ Standard Library

C++ programs consist of pieces called **classes** and **functions.** You can program each piece yourself, but most C++ programmers take advantage of the rich collections of classes and functions in the **C++ Standard Library**. Thus, there are really two parts to learning the C++ "world." The first is learning the C++ language itself (often referred to as the "core language"); the second is learning how to use the classes and functions in the C++ Standard Library. We discuss many of these classes and functions. P. J. Plauger's book, *The Standard C Library* (Upper Saddle River, NJ: Prentice Hall PTR, 1992), is a must-read for program-

mers who need a deep understanding of the ANSI C library functions included in C++. Many special-purpose class libraries are supplied by independent software vendors.

Software Engineering Observation 1.1

Use a "building-block" approach to create programs. Avoid reinventing the wheel. Use existing pieces wherever possible. Called software reuse, this practice is central to effective object-oriented programming.

Software Engineering Observation 1.2

When programming in C++, you typically will use the following building blocks: classes and functions from the C++ Standard Library, classes and functions you and your colleagues create, and classes and functions from various popular third-party libraries.

The advantage of creating your own functions and classes is that you'll know exactly how they work. You'll be able to examine the C++ code. The disadvantage is the time-consuming and complex effort that goes into designing, developing and maintaining new functions and classes that are correct and operate efficiently.

Performance Tip 1.2

Using C++ Standard Library functions and classes instead of writing your own versions can improve program performance, because they're written carefully to perform efficiently. This technique also shortens program development time.

Portability Tip 1.1

Using C++ Standard Library functions and classes instead of writing your own improves program portability, because they're included in every C++ implementation.

1.7 Programming Languages

In this section, we provide brief comments on several popular programming languages (Fig. 1.5).

Programming language	Description
Fortran	Fortran (FORmula TRANslator) was developed by IBM Corporation in the mid-1950s to be used for scientific and engineering applications that require complex mathematical computations. It's still widely used and its latest versions support object-oriented programming.
COBOL	COBOL (COmmon Business Oriented Language) was developed in the late 1950s by computer manufacturers, the U.S. government and industrial computer users, based on a language developed by Grace Hopper, a career U.S. Navy officer and computer scientist. COBOL is still widely used for commercial applications that require precise and efficient manipulation of large amounts of data. Its latest version supports object-oriented programming.

Fig. 1.5 | Some other programming languages. (Part 1 of 3.)

Programming language	Description
Pascal	Research in the 1960s resulted in *structured programming*—a disciplined approach to writing programs that are clearer, easier to test and debug and easier to modify than programs produced with previous techniques. The Pascal language developed by Professor Niklaus Wirth in 1971 grew out of this research. It was popular for teaching structured programming for several decades.
Ada	Ada, based on Pascal, was developed under the sponsorship of the U.S. Department of Defense (DOD) during the 1970s and early 1980s. The DOD wanted a single language that would fill most of its needs. The Pascal-based language was named after Lady Ada Lovelace, daughter of the poet Lord Byron. She's credited with writing the world's first computer program in the early 1800s (for the Analytical Engine mechanical computing device designed by Charles Babbage). Ada also supports object-oriented programming.
Basic	Basic was developed in the 1960s at Dartmouth College to familiarize novices with programming techniques. Many of its latest versions are object oriented.
Objective-C	Objective-C is an object-oriented language based on C. It was developed in the early 1980s and later acquired by NeXT, which in turn was acquired by Apple. It became the key programming language for the OS X operating system and all iOS-powered devices (such as iPods, iPhones and iPads).
Swift	Swift, which was introduced in 2014, is Apple's programming language of the future for developing iOS and OS X applications (apps). Swift is a contemporary language that includes popular programming-language features from languages such as Objective-C, Java, C#, Ruby, Python and others. In 2015, Apple released Swift 2 with new and updated features. According to the Tiobe Index, Swift has already become one of the most popular programming languages. Swift is now *open source* (Section 1.11.2), so it can be used on non-Apple platforms as well.
Java	Sun Microsystems in 1991 funded an internal corporate research project led by James Gosling, which resulted in the C++-based object-oriented programming language called Java. A key goal of Java is to enable developers to write programs that will run on a great variety of computer systems and computer-controlled devices. This is sometimes called "write once, run anywhere." Java is used to develop large-scale enterprise applications, to enhance the functionality of web servers (the computers that provide the content we see in our web browsers), to provide applications for consumer devices (e.g., smartphones, tablets, television set-top boxes, appliances, automobiles and more) and for many other purposes. Java is also the key language for developing Android smartphone and tablet apps.
Visual Basic	Microsoft's Visual Basic language was introduced in the early 1990s to simplify the development of Microsoft Windows applications. Its latest versions support object-oriented programming.
C#	Microsoft's three primary object-oriented programming languages are C# (based on C++ and Java), Visual C++ (based on C++) and Visual Basic (based on the original Basic). C# was developed to integrate the web into computer applications, and is now widely used to develop enterprise applications and for mobile application development.

Fig. 1.5 | Some other programming languages. (Part 2 of 3.)

Programming language	Description
PHP	PHP is an object-oriented, *open-source* (see Section 1.11.2) "scripting" language supported by a community of developers and used by numerous websites. PHP is platform independent—implementations exist for all major UNIX, Linux, Mac and Windows operating systems.
Python	Python, another object-oriented scripting language, was released publicly in 1991. Developed by Guido van Rossum of the National Research Institute for Mathematics and Computer Science in Amsterdam (CWI), Python draws heavily from Modula-3—a systems programming language. Python is "extensible"—it can be extended through classes and programming interfaces.
JavaScript	JavaScript is the most widely used scripting language. It's primarily used to add programmability to web pages—for example, animations and interactivity with the user. It's provided with all major web browsers.
Ruby on Rails	Ruby—created in the mid-1990s by Yukihiro Matsumoto—is an open-source, object-oriented programming language with a simple syntax that's similar to Python. Ruby on Rails combines the scripting language Ruby with the Rails web application framework developed by the company 37Signals. Their book, *Getting Real* (`http://gettingreal.37signals.com/toc.php`), is a must read for web developers. Many Ruby on Rails developers have reported productivity gains over other languages when developing database-intensive web applications.
Scala	Scala (`www.scala-lang.org/node/273`)—short for "scalable language"—was designed by Martin Odersky, a professor at École Polytechnique Fédérale de Lausanne (EPFL) in Switzerland. Released in 2003, Scala uses both the object-oriented programming and functional programming paradigms and is designed to integrate with Java. Programming in Scala can reduce the amount of code in your applications significantly.

Fig. 1.5 | Some other programming languages. (Part 3 of 3.)

1.8 Introduction to Object Technology

Building software quickly, correctly and economically remains an elusive goal at a time when demands for new and more powerful software are soaring. *Objects*, or more precisely—as we'll see in Chapter 3—the *classes* objects come from, are essentially *reusable* software components. There are date objects, time objects, audio objects, video objects, automobile objects, people objects, etc. Almost any *noun* can be reasonably represented as a software object in terms of *attributes* (e.g., name, color and size) and *behaviors* (e.g., calculating, moving and communicating). Software developers have discovered that using a modular, object-oriented design-and-implementation approach can make software development groups much more productive than was possible with earlier techniques—object-oriented programs are often easier to understand, correct and modify.

The Automobile as an Object

Let's begin with a simple analogy. Suppose you want to *drive a car and make it go faster by pressing its accelerator pedal*. What must happen before you can do this? Well, before you can drive a car, someone has to *design* it. A car typically begins as engineering drawings,

similar to the *blueprints* that describe the design of a house. These drawings include the design for an accelerator pedal. The pedal *hides* from the driver the complex mechanisms that actually make the car go faster, just as the brake pedal hides the mechanisms that slow the car, and the steering wheel *hides* the mechanisms that turn the car. This enables people with little or no knowledge of how engines, braking and steering mechanisms work to drive a car easily.

Before you can drive a car, it must be *built* from the engineering drawings that describe it. A completed car has an *actual* accelerator pedal to make the car go faster, but even that's not enough—the car won't accelerate on its own (hopefully!), so the driver must *press* the pedal to accelerate the car.

Functions, Member Functions and Classes

Let's use our car example to introduce some key object-oriented programming concepts. Performing a task in a program requires a **function**. The function houses the program statements that actually perform its task. It *hides* these statements from its user, just as the accelerator pedal of a car hides from the driver the mechanisms of making the car go faster. In C++, we often create a program unit called a **class** to house the set of functions that perform the class's tasks—these are known as the class's **member functions**. For example, a class that represents a bank account might contain a member function to *deposit* money to an account, another to *withdraw* money from an account and a third to *query* what the account's current balance is. A class is similar to a car's engineering drawings, which house the design of an accelerator pedal, brake pedal, steering wheel, and so on.

Instantiation

Just as someone has to *build a car* from its engineering drawings before you can actually drive a car, you must *build an object* from a class before a program can perform the tasks that the class's member functions define. The process of doing this is called *instantiation*. An object is then referred to as an **instance** of its class.

Reuse

Just as a car's engineering drawings can be *reused* many times to build many cars, you can *reuse* a class many times to build many objects. Reuse of existing classes when building new classes and programs saves time and effort. Reuse also helps you build more reliable and effective systems, because existing classes and components often have gone through extensive *testing*, *debugging* and *performance tuning*. Just as the notion of *interchangeable parts* was crucial to the Industrial Revolution, reusable classes are crucial to the software revolution that has been spurred by object technology.

Messages and Member-Function Calls

When you drive a car, pressing its gas pedal sends a *message* to the car to perform a task—that is, to go faster. Similarly, you *send messages to an object*. Each message is implemented as a **member-function call** that tells a member function of the object to perform its task. For example, a program might call a particular bank-account object's *deposit* member function to increase the account's balance.

Attributes and Data Members

A car, besides having capabilities to accomplish tasks, also has *attributes*, such as its color, its number of doors, the amount of gas in its tank, its current speed and its record of total

miles driven (i.e., its odometer reading). Like its capabilities, the car's attributes are represented as part of its design in its engineering diagrams (which, for example, include an odometer and a fuel gauge). As you drive an actual car, these attributes are carried along with the car. Every car maintains its *own* attributes. For example, each car knows how much gas is in its own gas tank, but not how much is in the tanks of *other* cars.

An object, similarly, has attributes that it carries along as it's used in a program. These attributes are specified as part of the object's class. For example, a bank-account object has a *balance attribute* that represents the amount of money in the account. Each bank-account object knows the balance in the account it represents, but *not* the balances of the *other* accounts in the bank. Attributes are specified by the class's **data members**.

Encapsulation

Classes **encapsulate** (i.e., wrap) attributes and member functions into objects created from those classes—an object's attributes and member functions are intimately related. Objects may communicate with one another, but they're normally not allowed to know how other objects are implemented—implementation details are *hidden* within the objects themselves. This **information hiding**, as we'll see, is crucial to good software engineering.

Inheritance

A new class of objects can be created quickly and conveniently by **inheritance**—the new class absorbs the characteristics of an existing class, possibly customizing them and adding unique characteristics of its own. In our car analogy, an object of class "convertible" certainly *is an* object of the more *general* class "automobile," but more *specifically*, the roof can be raised or lowered.

Object-Oriented Analysis and Design (OOAD)

Soon you'll be writing programs in C++. How will you create the **code** (i.e., the program instructions) for your programs? Perhaps, like many programmers, you'll simply turn on your computer and start typing. This approach may work for small programs (like the ones we present in the early chapters of the book), but what if you were asked to create a software system to control thousands of automated teller machines for a major bank? Or suppose you were asked to work on a team of thousands of software developers building the next generation of the U.S. air traffic control system? For projects so large and complex, you should not simply sit down and start writing programs.

To create the best solutions, you should follow a detailed **analysis** process for determining your project's **requirements** (i.e., defining *what* the system is supposed to do) and developing a **design** that satisfies them (i.e., deciding *how* the system should do it). Ideally, you'd go through this process and carefully review the design (and have your design reviewed by other software professionals) before writing any code. If this process involves analyzing and designing your system from an object-oriented point of view, it's called an **object-oriented analysis and design (OOAD) process**. Languages like C++ are object oriented. Programming in such a language, called **object-oriented programming (OOP)**, allows you to implement an object-oriented design as a working system.

The UML (Unified Modeling Language)

Although many different OOAD processes exist, a single graphical language for communicating the results of *any* OOAD process has come into wide use. This language, known as the Unified Modeling Language (UML), is now the most widely used graphical scheme

for modeling object-oriented systems. We present our first UML diagrams in Chapters 3 and 4, then use them in our deeper treatment of object-oriented programming through Chapter 12. In our *optional* ATM Software Engineering Case Study in Chapters 25–26 we present a simple subset of the UML's features as we guide you through an object-oriented design and implementation experience.

1.9 Typical C++ Development Environment

C++ systems generally consist of three parts: a program development environment, the language and the C++ Standard Library. C++ programs typically go through six phases: edit, preprocess, compile, link, load and execute. The following discussion explains a typical C++ program development environment.

Phase 1: Editing a Program

Phase 1 consists of editing a file with an *editor program*, normally known simply as an *editor* (Fig. 1.6). You type a C++ program (typically referred to as **source code**) using the editor, make any necessary corrections and save the program on your computer's disk. C++ source code filenames often end with the `.cpp`, `.cxx`, `.cc` or `.C` (uppercase) extensions which indicate that a file contains C++ source code. See the documentation for your C++ compiler for more information on filename extensions. Two editors widely used on Linux systems are `vim` and `emacs`. You can also use a simple text editor, such as Notepad in Windows, to write your C++ code.

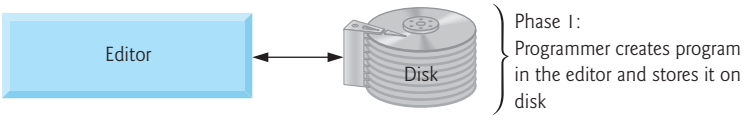


Fig. 1.6 | Typical C++ development environment—editing phase.

Integrated development environments (IDEs) are available from many major software suppliers. IDEs provide tools that support the software development process, including editors for writing and editing programs and debuggers for locating **logic errors**—errors that cause programs to execute incorrectly. Popular IDEs include Microsoft® Visual Studio 2015 Community Edition, NetBeans, Eclipse, Apple's Xcode®, CodeLite and Clion.

Phase 2: Preprocessing a C++ Program

In Phase 2, you give the command to **compile** the program (Fig. 1.7). In a C++ system, a **preprocessor** program executes automatically before the compiler's translation phase begins (so we call preprocessing Phase 2 and compiling Phase 3). The C++ preprocessor obeys commands called **preprocessing directives**, which indicate that certain manipulations are to be performed on the program before compilation. These manipulations usually include (i.e., copy into the program file) other text files to be compiled, and perform various text replacements. The most common preprocessing directives are discussed in the early chapters; a detailed discussion of preprocessor features appears in Appendix E, Preprocessor.

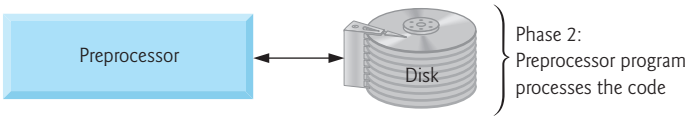


Fig. 1.7 | Typical C++ development environment—preprocessor phase.

Phase 3: Compiling a C++ Program

In Phase 3, the compiler translates the C++ program into machine-language code—also referred to as **object code** (Fig. 1.8).

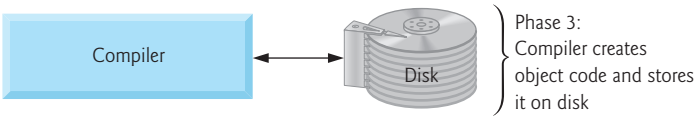


Fig. 1.8 | Typical C++ development environment—compilation phase.

Phase 4: Linking

Phase 4 is called **linking.** C++ programs typically contain references to functions and data defined elsewhere, such as in the standard libraries or in the private libraries of groups of programmers working on a particular project (Fig. 1.9). The object code produced by the C++ compiler typically contains "holes" due to these missing parts. A **linker** links the object code with the code for the missing functions to produce an **executable program** (with no missing pieces). If the program compiles and links correctly, an executable image is produced.

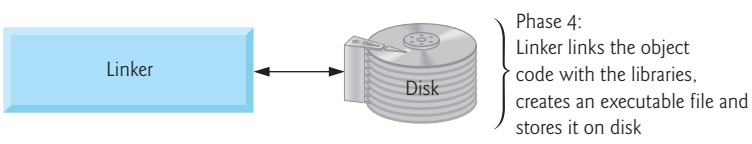


Fig. 1.9 | Typical C++ development environment—linking phase.

Phase 5: Loading

Phase 5 is called **loading.** Before a program can be executed, it must first be placed in memory (Fig. 1.10). This is done by the **loader,** which takes the executable image from disk and transfers it to memory. Additional components from shared libraries that support the program are also loaded.

Phase 6: Execution

Finally, the computer, under the control of its CPU, **executes** the program one instruction at a time (Fig. 1.11). Some modern computer architectures often execute several instructions in parallel.

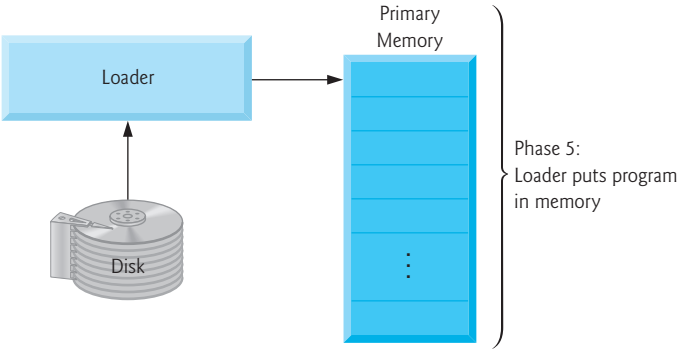


Fig. 1.10 | Typical C++ development environment—loading phase.

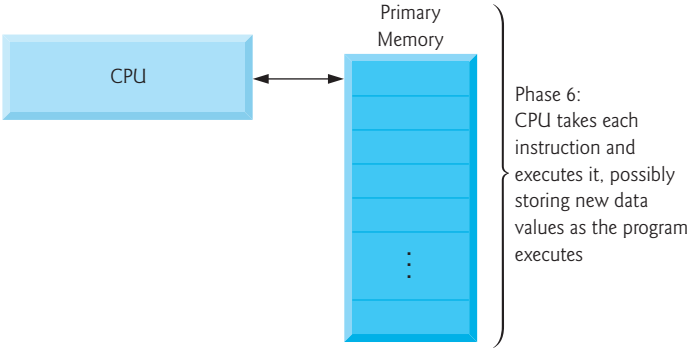


Fig. 1.11 | Typical C++ development environment—execution phase.

Problems That May Occur at Execution Time

Programs might not work on the first try. Each of the preceding phases can fail because of various errors that we'll discuss throughout this book. For example, an executing program might try to divide by zero (an illegal operation for integer arithmetic in C++). This would cause the C++ program to display an error message. If this occurred, you'd have to return to the edit phase, make the necessary corrections and proceed through the remaining phases again to determine that the corrections fixed the problem(s). [*Note:* Most programs in C++ input or output data.] Certain C++ functions take their input from `cin` (the **standard input stream**; pronounced "see-in"), which is normally the keyboard, but `cin` can be redirected to another device. Data is often output to `cout` (the **standard output stream**; pronounced "see-out"), which is normally the computer screen, but `cout` can be redirected to another device. When we say that a program prints a result, we normally mean that the result is displayed on a screen. Data may be output to other devices, such as disks, hard-copy printers or even transmitted over the Internet. There is also a **standard error stream** referred to as **`cerr`**. The `cerr` stream (normally connected to the screen) is used for displaying error messages.

Common Programming Error 1.1

Errors such as division by zero occur as a program runs, so they're called runtime errors or execution-time errors. Fatal runtime errors cause programs to terminate immediately without having successfully performed their jobs. Nonfatal runtime errors allow programs to run to completion, often producing incorrect results.

1.10 Test-Driving a C++ Application

In this section, you'll compile, run and interact with your first C++ application—an entertaining guess-the-number game, which picks a number from 1 to 1000 and prompts you to guess it. If your guess is correct, the game ends. If your guess is not correct, the application indicates whether your guess is higher or lower than the correct number. There is no limit on the number of guesses you can make. [*Note:* For this test drive only, we've modified this application from the exercise you'll be asked to create in Chapter 6, Functions and an Introduction to Recursion. Normally this application randomly selects the correct answer as you execute the program. The modified application uses the same correct answer every time the program executes (though this may vary by compiler), so you can use the *same* guesses we use in this section and see the *same* results as we walk you through interacting with your first C++ application.]

We'll demonstrate running a C++ application using

- Visual Studio 2015 Community Edition for Windows (Section 1.10.1)
- GNU C++ in a shell on Linux (Section 1.10.2)
- Clang/LLVM in Xcode on Mac OS X (Section 1.10.3).

The application runs similarly on all three platforms. You need to read only the section that corresponds to your operating system and compiler. Many development environments are available in which you can compile, build and run C++ applications—CodeLite, Clion, NetBeans and Eclipse are just a few. Consult your instructor or the online documentation for information on your specific development environment.

In the following steps, you'll run the application and enter various numbers to guess the correct number. The elements and functionality that you see in this application are typical of those you'll learn to program in this book. We use fonts to distinguish between features you see on the screen and elements that are not directly related to the screen. We emphasize screen features like titles and menus (e.g., the **File** menu) in a semibold **sans-serif bold** font and emphasize filenames, text displayed by an application and values you should enter into an application (e.g., `GuessNumber` or `500`) in a `sans-serif` font.

1.10.1 Compiling and Running an Application in Visual Studio 2015 for Windows

In this section, you'll run a C++ program on Windows using Microsoft Visual Studio 2015 Community Edition. We assume that you've already read the Before You Begin section for instructions on installing the IDE and downloading the book's code examples.

There are several versions of Visual Studio available—on some versions, the options, menus and instructions we present might differ slightly. From this point forward, we'll refer to Visual Studio 2015 Community Edition simply as "Visual Studio" or "the IDE."

Step 1: Checking Your Setup

It's important to read this book's Before You Begin section to make sure that you've installed Visual Studio and copied the book's examples to your hard drive correctly.

Step 2: Launching Visual Studio

Open Visual Studio from the **Start** menu. The IDE displays the **Start Page** (Fig. 1.12), which provides links for creating new programs, opening existing programs and learning about the IDE and various programming topics. Close this window for now by clicking the **X** in its tab—you can access this window any time by selecting **View > Start Page**. We use the **>** character to indicate selecting a menu item from a menu. For example, the notation **File > Open** indicates that you should select the **Open** menu item from the **File** menu.

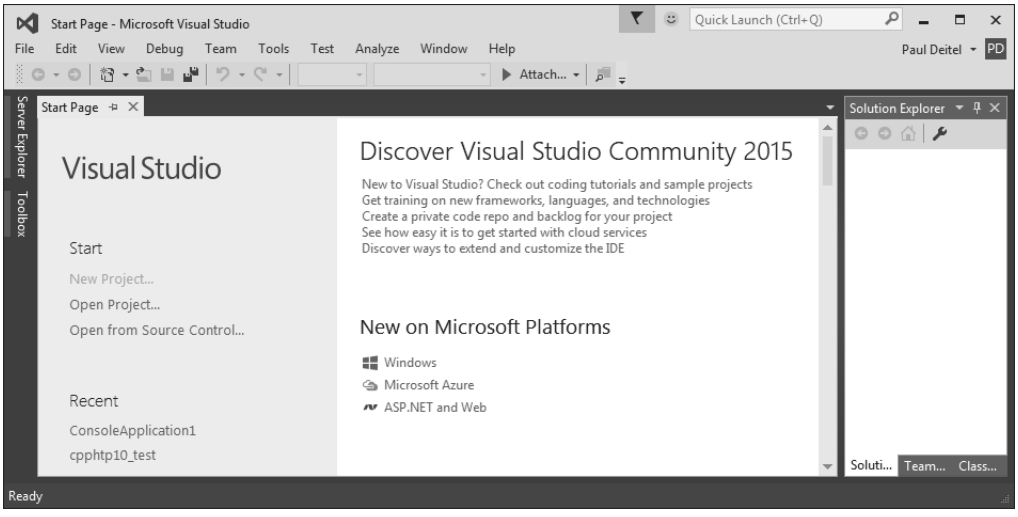


Fig. 1.12 | Visual Studio 2015 Community Edition window showing the **Start Page**.

Step 3: Creating a Project

A **project** is a group of related files, such as the C++ source-code files that compose an application. Visual Studio organizes applications into projects and **solutions**, which contain one or more projects. Multiple-project solutions are used to create large-scale applications. Each application in this book will be a solution containing a single project.

The Visual Studio projects we created for this book's examples are **Win32 Console Application** projects that you'll execute from the IDE. To create a project:

1. Select **File > New > Project...**.
2. At the **New Project** dialog's left side, select the category **Installed > Templates > Visual C++ > Win32** (Fig. 1.13).
3. In the **New Project** dialog's middle section, select **Win32 Console Application**.
4. Provide a name for your project in the **Name** field—we specified `Guess Number`—then click **OK** to display the **Win32 Application Wizard** window, then click **Next >** to display the **Application Settings** step.

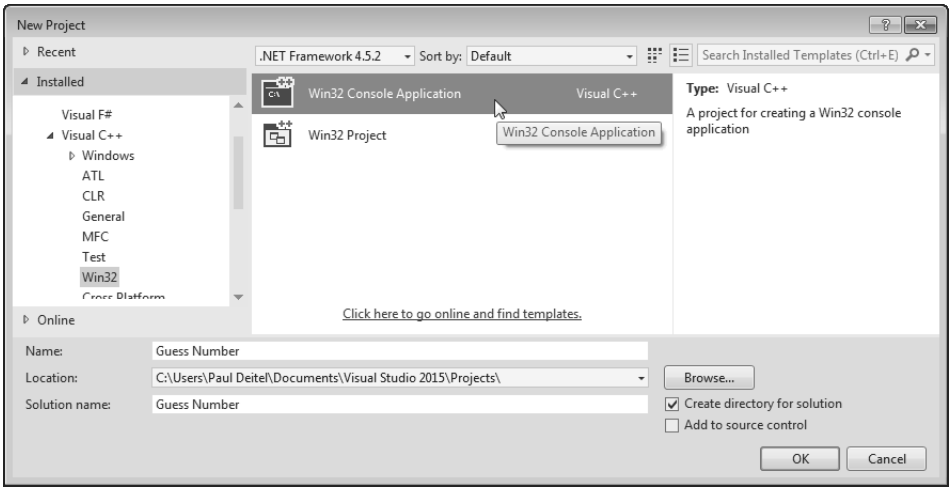


Fig. 1.13 | Visual Studio 2015 Community Edition **New Project** dialog.

5. Configure the settings as shown in Fig. 1.14 to create a solution containing an empty project, then click **Finish**.

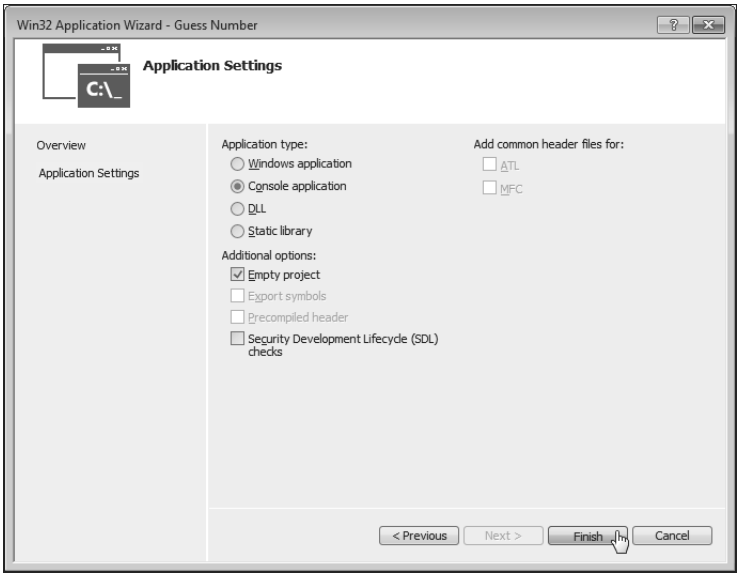


Fig. 1.14 | **Win32 Application Wizard** window's **Application Settings** step.

At this point, the IDE creates your project and places its folder in

```
C:\Users\YourUserAccount\Documents\Visual Studio 2015\Projects
```

then opens the window in Fig. 1.15. This window displays editors as tabbed windows (one for each file) when you're editing code. Also displayed is the **Solution Explorer** in which you can view and manage your application's files. In this book's examples, you'll typically place each program's code files in the **Source Files** folder. If the **Solution Explorer** is not displayed, you can display it by selecting **View > Solution Explorer**.

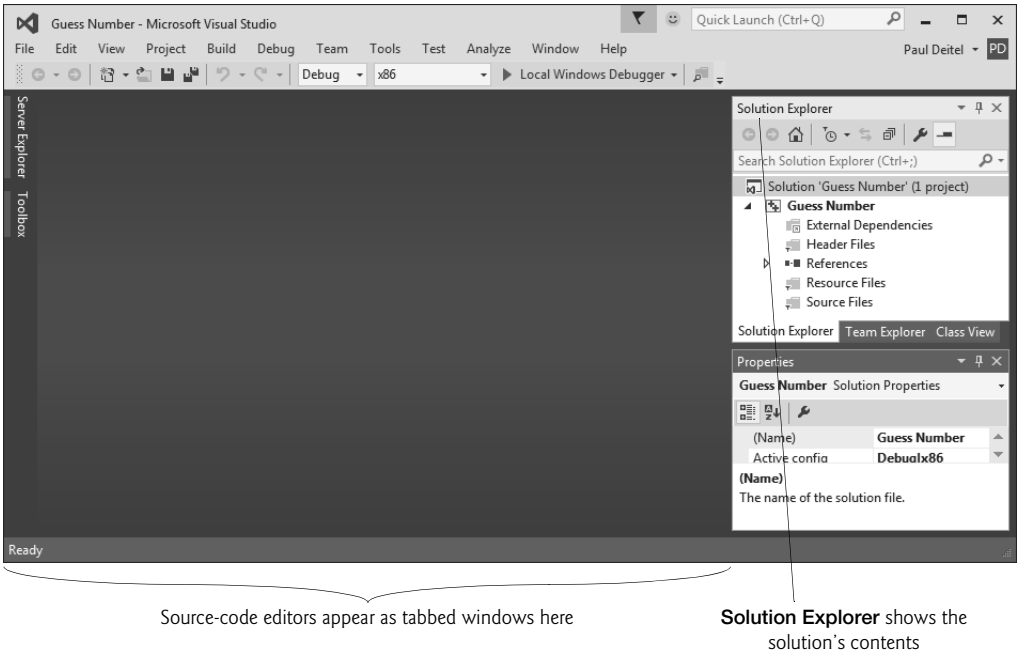


Fig. 1.15 | Visual Studio window after creating the `Guess Number` project.

Step 4: Adding the `GuessNumber.cpp` File into the Project

Next, you'll add `GuessNumber.cpp` to the project you created in *Step 3*. In Windows Explorer (Windows 7) or File Explorer (Windows 8 and 10), open the `ch01` folder in the book's `examples` folder, then drag `GuessNumber.cpp` onto the **Source Files** folder in the **Solution Explorer**.[10]

Step 5: Compiling and Running the Project

To compile and run the project so you can test-drive the application, select **Debug > Start without debugging** or simply type *Ctrl* + *F5*. If the program compiles correctly, the IDE opens a Command Prompt window and executes the program (Fig. 1.16)—we changed the Command Prompt's color scheme to make the screen captures more readable. The application displays `"Please type your first guess."`, then displays a question mark (?) as a prompt on the next line.

10. For the multiple source-code-file programs that you'll see beginning in Chapter 3, drag all the files for a given program to the **Source Files** folder. When you begin creating multiple source-code-file programs yourself, you can right click the **Source Files** folder and select **Add > New Item...** to display a dialog for adding a new file.

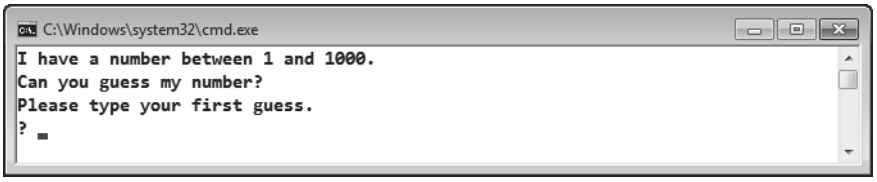


Fig. 1.16 | Command Prompt showing the running program.

Step 6: Entering Your First Guess
Type **500** and press *Enter*. The application displays `"Too high. Try again."` (Fig. 1.17), meaning that the value you entered is greater than the number the application chose as the correct guess.

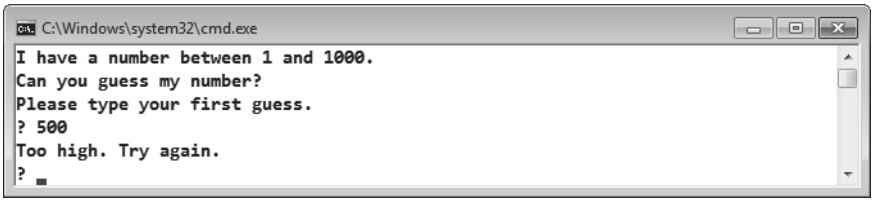


Fig. 1.17 | Entering an initial guess and receiving feedback.

Step 7: Entering Another Guess
At the next prompt, enter **250** (Fig. 1.18). The application displays `"Too high. Try again."`, because the value you entered once again is greater than the correct guess.

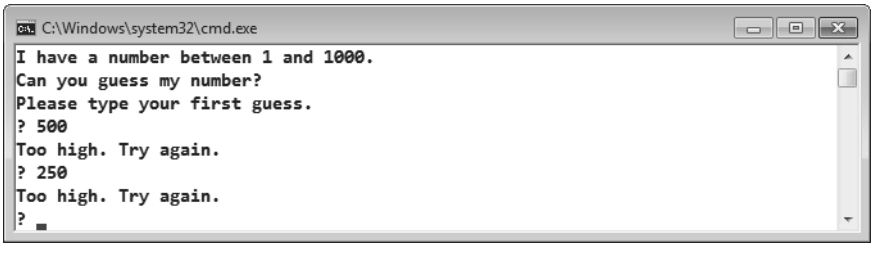


Fig. 1.18 | Entering a second guess and receiving feedback.

Step 8: Entering Additional Guesses
Continue to play the game (Fig. 1.19) by entering values until you guess the correct number. When you guess correctly, the application displays `"Excellent! You guessed the number."`

Step 9: Playing the Game Again or Exiting the Application
After you guess the correct number, the application asks if you'd like to play another game. At the `"Would you like to play again (y or n)?"` prompt, entering the one character **y**

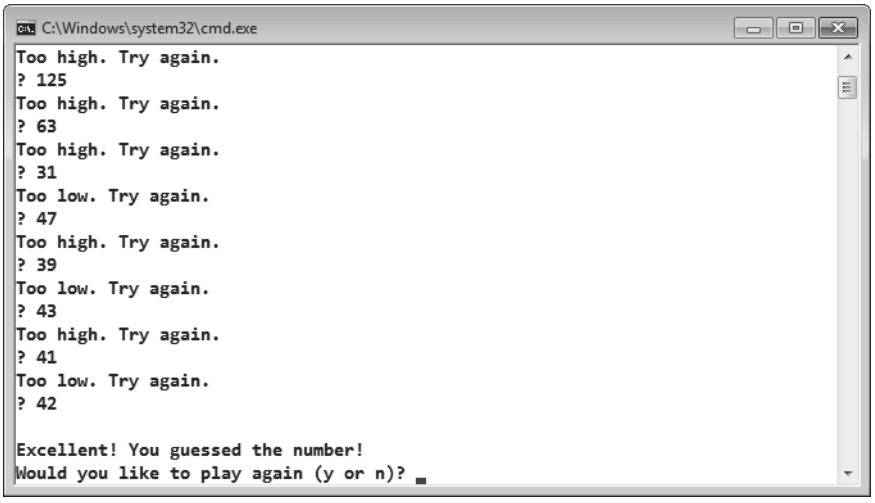


Fig. 1.19 | Entering additional guesses and guessing the correct number.

causes the application to choose a new number and displays the message `"Please type your first guess."` followed by a question-mark prompt so you can make your first guess in the new game. Entering the character **`n`** terminates the application. Each time you execute this application from the beginning (*Step 5*), it will choose the same numbers for you to guess.

1.10.2 Compiling and Running Using GNU C++ on Linux

For this test drive, we assume that you read the Before You Begin section and that you placed the downloaded examples in your home directory on your Linux system. Please see your instructor if you have any questions regarding copying the files to your home directory. In this section's figures, we use **`bold`** text to highlight the text that you type. The prompt in the shell on our system uses the tilde (~) character to represent the home directory, and each prompt ends with the dollar sign (`$`) character. The prompt will vary among Linux systems.

Step 1: Locating the Completed Application

From a Linux shell, use the command `cd` to change to the completed application directory (Fig. 1.20) by typing

```
cd examples/ch01
```

then pressing *Enter*.

```
~$ cd examples/ch01
~/examples/ch01$
```

Fig. 1.20 | Changing to the `GuessNumber` application's directory.

Step 2: Compiling the Application

Before running the application, you must first compile it (Fig. 1.21) by typing

```
g++ -std=c++14 GuessNumber.cpp -o GuessNumber
```

This command compiles the application for C++14 (the current C++ version) and produces an executable file called `GuessNumber`.

```
~/examples/ch01$ g++ -std=c++14 GuessNumber.cpp -o GuessNumber
~/examples/ch01$
```

Fig. 1.21 | Compiling the **GuessNumber** application using the `g++` command.

Step 3: Running the Application

To run the executable file `GuessNumber`, type `./GuessNumber` at the next prompt, then press *Enter* (Fig. 1.22). The `./` tells Linux to run from the current directory and is required to indicate that `GuessNumber` is an executable file.

```
~/examples/ch01$ ./GuessNumber
I have a number between 1 and 1000.
Can you guess my number?
Please type your first guess.
?
```

Fig. 1.22 | Running the **GuessNumber** application.

Step 4: Entering Your First Guess

The application displays `"Please type your first guess."`, then displays a question mark (?) as a prompt on the next line (Fig. 1.22). At the prompt, enter **500** (Fig. 1.23). [Note that the outputs may vary based on the compiler you're using.]

```
~/examples/ch01$ ./GuessNumber
I have a number between 1 and 1000.
Can you guess my number?
Please type your first guess.
? 500
Too high. Try again.
?
```

Fig. 1.23 | Entering an initial guess.

Step 5: Entering Another Guess

The application displays `"Too high. Try again."`, meaning that the value you entered is greater than the number the application chose as the correct guess (Fig. 1.23). At the next prompt, enter **250** (Fig. 1.24). This time the application displays `"Too low. Try again."`, because the value you entered is less than the correct guess.

```
~/examples/ch01$ ./GuessNumber
I have a number between 1 and 1000.
Can you guess my number?
Please type your first guess.
? 500
Too high. Try again.
? 250
Too low. Try again.
?
```

Fig. 1.24 | Entering a second guess and receiving feedback.

Step 6: Entering Additional Guesses
Continue to play the game (Fig. 1.25) by entering values until you guess the correct number. When you guess correctly, the application displays `"Excellent! You guessed the number."`

```
Too low. Try again.
? 375
Too low. Try again.
? 437
Too high. Try again.
? 406
Too high. Try again.
? 391
Too high. Try again.
? 383
Too low. Try again.
? 387
Too high. Try again.
? 385
Too high. Try again.
? 384
Excellent! You guessed the number.
Would you like to play again (y or n)?
```

Fig. 1.25 | Entering additional guesses and guessing the correct number.

Step 7: Playing the Game Again or Exiting the Application
After you guess the correct number, the application asks if you'd like to play another game. At the `"Would you like to play again (y or n)?"` prompt, entering the one character **`y`** causes the application to choose a new number and displays the message `"Please type your first guess."` followed by a question-mark prompt so you can make your first guess in the new game. Entering the character **`n`** ends the application, returns you to the shell and awaits your next command. Each time you execute this application from the beginning (i.e., *Step 3*), it will choose the same numbers for you to guess.

1.10.3 Compiling and Running with Xcode on Mac OS X

In this section, we present how to run a C++ program on a Mac OS X using Apple's Xcode IDE.

Step 1: Checking Your Setup

It's important to read this book's Before You Begin section to make sure that you've installed Apple's Xcode IDE and copied the book's examples to your hard drive correctly.

Step 2: Launching Xcode

Open a Finder window, select **Applications** and double click the Xcode icon ( ). If this is your first time running Xcode, the **Welcome to Xcode** window will appear (Fig. 1.26). Close this window for now—you can access it any time by selecting **Window > Welcome to Xcode**. We use the **>** character to indicate selecting a menu item from a menu. For example, the notation **File > Open...** indicates that you should select the **Open...** menu item from the **File** menu.

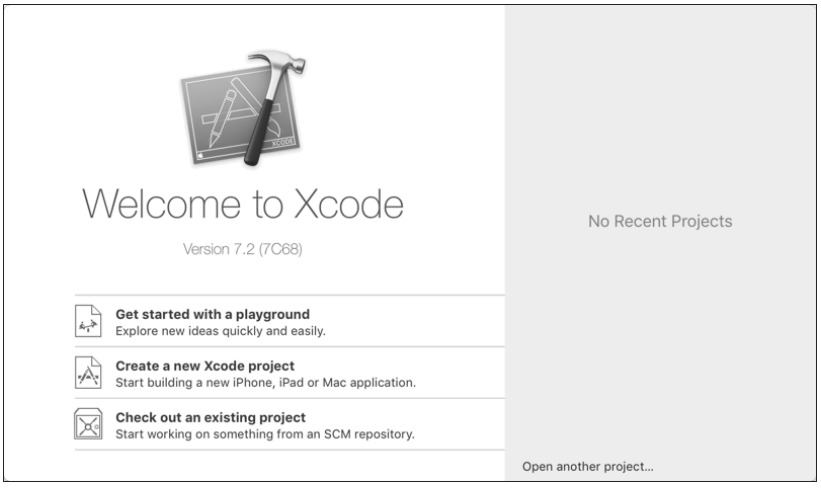


Fig. 1.26 | **Welcome to Xcode** window.

Step 3: Creating a Project

A **project** is a group of related files, such as the C++ source-code files that compose an application. The Xcode projects we created for this book's examples are **OS X Command Line Tool** projects that you'll execute directly in the IDE. To create a project:

1. Select **File > New > Project...**.
2. In the **OS X** subcategory **Application**, select **Command Line Tool** and click **Next**.
3. Provide a name for your project in the **Product Name** field—we specified `Guess Number`.
4. Ensure that the selected **Language** is **C++** and click **Next**.
5. Specify where you want to store your project, then click **Create**. (See the Before You Begin section for information on configuring a project to use C++14.)

Figure 1.27 shows the **workspace window** that appears after you create the project. By default, Xcode creates a `main.cpp` source-code file containing a simple program that displays `"Hello, World!"`. The window is divided into four main areas below the toolbar: the

Navigator area, **Editor** area and **Utilities** area are displayed initially. We'll explain momentarily how to display the **Debug** area in which you'll run and interact with the program.

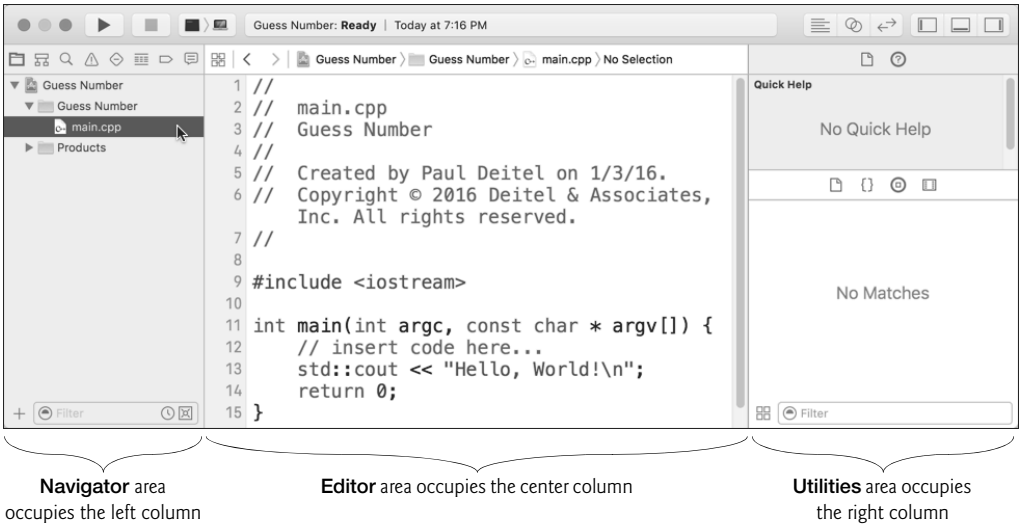


Fig. 1.27 | Sample Xcode C++ project with `main.cpp` selected.

At the left of the workspace window is the **Navigator** area, which has icons at its top for the *navigators* that can be displayed there. For this book, you'll primarily work with

- **Project** ( )—Shows all the files and folders in your project.
- **Issue** ( )—Shows you warnings and errors generated by the compiler.

You choose which navigator to display by clicking the corresponding button above the **Navigator** area of the window.

To the right of the **Navigator** area is the **Editor** area for editing source code. This area is always displayed in your workspace window. When you select a file in the **Project** navigator, the file's contents are displayed in the **Editor** area. At the right side of the workspace window is the **Utilities** area, which you will not use in this book. The **Debug** area, when displayed, appears below the **Editor** area.

The toolbar contains options for executing a program (Fig. 1.28(a)), a display area (Fig. 1.28(b)) to shows the progress of tasks executing in Xcode (such as the compilation status) and buttons (Fig. 1.28(c)) for hiding and showing areas in the workspace window.

a) The left side of the toolbar

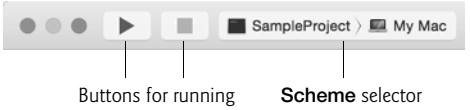


Fig. 1.28 | Xcode 7 toolbar. (Part 1 of 2.)

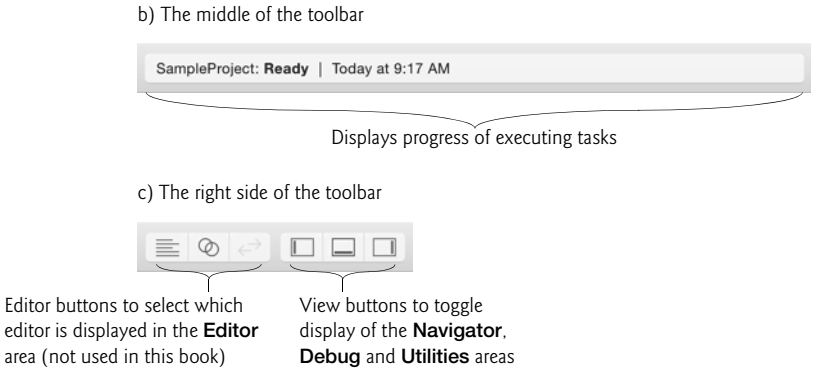


Fig. 1.28 | Xcode 7 toolbar. (Part 2 of 2.)

Step 4: Deleting the `main.cpp` File from the Project

You won't use `main.cpp` in this test-drive, so you should delete the file. In the **Project** navigator, right click the `main.cpp` file and select **Delete**. In the dialog that appears, select **Move to Trash** to delete the file from your system—the file will not be removed completely until you empty your trash.

Step 5: Adding the `GuessNumber.cpp` File into the Project

Next, you'll add `GuessNumber.cpp` to the project you created in *Step 3*. In a Finder window, open the `ch01` folder in the book's `examples` folder, then drag `GuessNumber.cpp` onto the **Guess Number** folder in the **Project** navigator. In the dialog that appears, ensure that **Copy items if needed** is checked, then click **Finish**.[11]

Step 6: Compiling and Running the Project

To compile and run the project so you can test-drive the application, simply click the run (▶) button at the left side of Xcode's toolbar. If the program compiles correctly, Xcode opens the **Debug** area (at the bottom of the **Editor** area) and executes the program in the right half of the **Debug** area (Fig. 1.29). The application displays `"Please type your first guess."`, then displays a question mark (?) as a prompt on the next line.

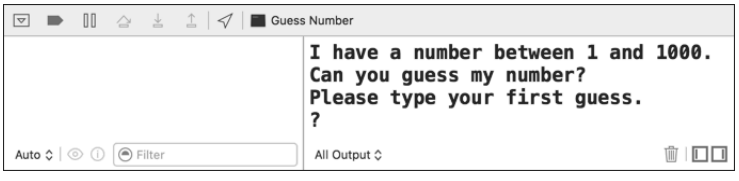


Fig. 1.29 | Debug area showing the running program.

11. For the multiple source-code-file programs that you'll see beginning in Chapter 3, drag all the files for a given program to the project's folder. When you begin creating programs with multiple source-code files, you can right click the project's folder and select **New File...** to display a dialog for adding a new file.

Step 7: Entering Your First Guess
Click in the **Debug** area, then type **500** and press *Return*. The application displays `"Too low. Try again."` (Fig. 1.30), meaning that the value you entered is less than the number the application chose as the correct guess.

```
I have a number between 1 and 1000.
Can you guess my number?
Please type your first guess.
? 500
Too low. Try again.
?
```

Fig. 1.30 | Entering an initial guess and receiving feedback.

Step 8: Entering Another Guess
At the next prompt, enter **750** (Fig. 1.31). The application displays `"Too low. Try again."`, because the value you entered once again is less than the correct guess.

```
I have a number between 1 and 1000.
Can you guess my number?
Please type your first guess.
? 500
Too low. Try again.
? 750
Too low. Try again.
?
```

Fig. 1.31 | Entering a second guess and receiving feedback.

Step 9: Entering Additional Guesses
Continue to play the game (Fig. 1.32) by entering values until you guess the correct number. When you guess correctly, the application displays `"Excellent! You guessed the number."`

```
Too low. Try again.
? 875
Too high. Try again.
? 812
Too high. Try again.
? 781
Too low. Try again.
? 797
Too low. Try again.
? 805
Too low. Try again.
? 808

Excellent! You guessed the number!
Would you like to play again (y or n)?
```

Fig. 1.32 | Entering additional guesses and guessing the correct number.

Playing the Game Again or Exiting the Application
After you guess the correct number, the application asks if you'd like to play another game. At the `"Would you like to play again (y or n)?"` prompt, entering the character **`y`** causes the application to choose a new number and displays the message `"Please type your first guess."` followed by a question-mark prompt so you can make your first guess in the new game. Entering the character **`n`** terminates the application. Each time you execute this application from the beginning (*Step 6*), it will choose the same numbers for you to guess.

1.11 Operating Systems

Operating systems are software systems that make using computers more convenient for users, application developers and system administrators. They provide services that allow each application to execute safely, efficiently and *concurrently* (i.e., in parallel) with other applications. The software that contains the core components of the operating system is called the **kernel**. Popular desktop operating systems include Linux, Windows and OS X (formerly called Mac OS X)—we used all three in developing this book. Popular mobile operating systems used in smartphones and tablets include Google's Android, Apple's iOS (for iPhone, iPad and iPod Touch devices) and Windows 10 Mobile. You can develop applications in C++ for all of these operating systems.

1.11.1 Windows—A Proprietary Operating System

In the mid-1980s, Microsoft developed the **Windows operating system**, consisting of a graphical user interface built on top of DOS (Disk Operating System)—an enormously popular personal-computer operating system that users interacted with by typing commands. Windows borrowed from many concepts (such as icons, menus and windows) developed by Xerox PARC and popularized by early Apple Macintosh operating systems. Windows 10 is Microsoft's latest operating system—its features include enhancements to the **Start** menu and user interface, Cortana personal assistant for voice interactions, Action Center for receiving notifications, Microsoft's new Edge web browser, and more. Windows is a *proprietary* operating system—it's controlled by Microsoft exclusively. Windows is by far the world's most widely used desktop operating system.

1.11.2 Linux—An Open-Source Operating System

The Linux operating system is perhaps the greatest success of the *open-source* movement. **Open-source software** departs from the *proprietary* software development style that dominated software's early years. With open-source development, individuals and companies *contribute* their efforts in developing, maintaining and evolving software in exchange for the right to use that software for their own purposes, typically at *no charge*. Open-source code is often scrutinized by a much larger audience than proprietary software, so errors often get removed faster. Open source also encourages innovation. Enterprise systems companies, such as IBM, Oracle and many others, have made significant investments in Linux open-source development.

Some key organizations in the open-source community are

- the Eclipse Foundation (the Eclipse Integrated Development Environment helps programmers conveniently develop software)
- the Mozilla Foundation (creators of the Firefox web browser)

- the Apache Software Foundation (creators of the Apache web server used to develop web-based applications)
- GitHub (which provides tools for managing open-source projects—it has millions of them under development).

Rapid improvements to computing and communications, decreasing costs and open-source software have made it much easier and more economical to create a software-based business now than just a decade ago. A great example is Facebook, which was launched from a college dorm room and built with open-source software.

The **Linux** kernel is the core of the most popular open-source, freely distributed, full-featured operating system. It's developed by a loosely organized team of volunteers and is popular in servers, personal computers and embedded systems (such as the computer systems at the heart of smartphones, smart TVs and automobile systems). Unlike that of proprietary operating systems like Microsoft's Windows and Apple's OS X, Linux source code (the program code) is available to the public for examination and modification and is free to download and install. As a result, Linux users benefit from a huge community of developers actively debugging and improving the kernel, and the ability to customize the operating system to meet specific needs.

A variety of issues—such as Microsoft's market power, the small number of user-friendly Linux applications and the diversity of Linux distributions, such as Red Hat Linux, Ubuntu Linux and many others—have prevented widespread Linux use on desktop computers. Linux has become extremely popular on servers and in embedded systems, such as Google's Android-based smartphones.

1.11.3 Apple's OS X; Apple's iOS for iPhone®, iPad® and iPod Touch® Devices

Apple, founded in 1976 by Steve Jobs and Steve Wozniak, quickly became a leader in personal computing. In 1979, Jobs and several Apple employees visited Xerox PARC (Palo Alto Research Center) to learn about Xerox's desktop computer that featured a graphical user interface (GUI). That GUI served as the inspiration for the Apple Macintosh, launched with much fanfare in a memorable Super Bowl ad in 1984.

The Objective-C programming language, created by Brad Cox and Tom Love at Stepstone in the early 1980s, added capabilities for object-oriented programming (OOP) to the C programming language. Steve Jobs left Apple in 1985 and founded NeXT Inc. In 1988, NeXT licensed Objective-C from StepStone and developed an Objective-C compiler and libraries which were used as the platform for the NeXTSTEP operating system's user interface, and Interface Builder—used to construct graphical user interfaces.

Jobs returned to Apple in 1996 when Apple bought NeXT. Apple's OS X operating system is a descendant of NeXTSTEP. Apple's proprietary operating system, **iOS**, is derived from Apple's OS X and is used in the iPhone, iPad and iPod Touch devices. In 2014, Apple introduced its new Swift programming language, which became open source in 2015. The iOS app-development community is gradually shifting from Objective-C to Swift.

1.11.4 Google's Android

Android—the fastest growing mobile and smartphone operating system—is based on the Linux kernel and Java. Android apps can also be developed in C++ and C. One benefit of

developing Android apps is the openness of the platform. The operating system is open source and free.

The Android operating system was developed by Android, Inc., which was acquired by Google in 2005. In 2007, the Open Handset Alliance™

```
http://www.openhandsetalliance.com/oha_members.html
```

was formed to develop, maintain and evolve Android, driving innovation in mobile technology and improving the user experience while reducing costs. According to IDC, after the first six months of 2015, Android had 82.8% of the global smartphone market share, compared to 13.9% for Apple, 2.6% for Microsoft and 0.3% for Blackberry.[12] The Android operating system is used in numerous smartphones, e-reader devices, tablets, in-store touch-screen kiosks, cars, robots, multimedia players and more. There are now more than 1.4 billion Android users.[13]

1.12 The Internet and the World Wide Web

In the late 1960s, ARPA—the Advanced Research Projects Agency of the United States Department of Defense—rolled out plans for networking the main computer systems of approximately a dozen ARPA-funded universities and research institutions. The computers were to be connected with communications lines operating at speeds on the order of 50,000 bits per second, a stunning rate at a time when most people (of the few who even had networking access) were connecting over telephone lines to computers at a rate of 110 bits per second. Academic research was about to take a giant leap forward. ARPA proceeded to implement what quickly became known as the ARPANET, the precursor to today's **Internet**. Today's fastest Internet speeds are on the order of billions of bits per second with trillion-bits-per-second speeds on the horizon!

Things worked out differently from the original plan. Although the ARPANET enabled researchers to network their computers, its main benefit proved to be the capability for quick and easy communication via what came to be known as electronic mail (e-mail). This is true even on today's Internet, with e-mail, instant messaging, file transfer and social media such as Facebook and Twitter enabling billions of people worldwide to communicate quickly and easily.

The protocol (set of rules) for communicating over the ARPANET became known as the **Transmission Control Protocol (TCP)**. TCP ensured that messages, consisting of sequentially numbered pieces called *packets*, were properly routed from sender to receiver, arrived intact and were assembled in the correct order.

The Internet: A Network of Networks

In parallel with the early evolution of the Internet, organizations worldwide were implementing their own networks for both intraorganization (that is, within an organization) and interorganization (that is, between organizations) communication. A huge variety of networking hardware and software appeared. One challenge was to enable these different

12. http://www.idc.com/prodserv/smartphone-os-market-share.jsp.
13. http://www.techtimes.com/articles/90028/20151002/google-says-android-has-more-than-1-4-billion-active-users-worldwide-with-300-million-on-lollipop.htm.

networks to communicate with each other. ARPA accomplished this by developing the Internet Protocol (IP), which created a true "network of networks," the current architecture of the Internet. The combined set of protocols is now called **TCP/IP**.

Businesses rapidly realized that by using the Internet, they could improve their operations and offer new and better services to their clients. Companies started spending large amounts of money to develop and enhance their Internet presence. This generated fierce competition among communications carriers and hardware and software suppliers to meet the increased infrastructure demand. As a result, **bandwidth**—the information-carrying capacity of communications lines—on the Internet has increased tremendously, while hardware costs have plummeted.

The World Wide Web: Making the Internet User-Friendly

The **World Wide Web** (simply called "the web") is a collection of hardware and software associated with the Internet that allows computer users to locate and view multimedia-based documents (documents with various combinations of text, graphics, animations, audios and videos) on almost any subject. The introduction of the web was a relatively recent event. In 1989, Tim Berners-Lee of CERN (the European Organization for Nuclear Research) began to develop a technology for sharing information via "hyperlinked" text documents. Berners-Lee called his invention the **HyperText Markup Language (HTML)**. He also wrote communication protocols such as **HyperText Transfer Protocol (HTTP)** to form the backbone of his new hypertext information system, which he referred to as the World Wide Web.

In 1994, Berners-Lee founded the **World Wide Web Consortium** (**W3C**, `http://www.w3.org`), devoted to developing web technologies. One of the W3C's primary goals is to make the web universally accessible to everyone regardless of disabilities, language or culture.

Web Services

Web services are software components stored on one computer that can be accessed by an app (or other software component) on another computer over the Internet. With web services, you can create *mashups*, which enable you to rapidly develop apps by combining complementary web services, often from multiple organizations and possibly other forms of information feeds. For example, 100 Destinations (`http://www.100destinations.co.uk`) combines the photos and tweets from Twitter with the mapping capabilities of Google Maps to allow you to explore countries around the world through the photos of others.

Programmableweb (`http://www.programmableweb.com/`) provides a directory of over 11,150 APIs and 7,300 mashups, plus how-to guides and sample code for creating your own mashups. According to Programmableweb, the three most widely used APIs for mashups are Google Maps, Twitter and YouTube.

Ajax

Ajax technology helps Internet-based applications perform like desktop applications—a difficult task, given that such applications suffer transmission delays as data is shuttled back and forth between your computer and server computers on the Internet. Using Ajax,

applications like Google Maps have achieved excellent performance and approach the look-and-feel of desktop applications.

The Internet of Things

The Internet is no longer just a network of computers—it's an **Internet of Things**. A *thing* is any object with an IP address and the ability to send data automatically over the Internet—e.g., a car with a transponder for paying tolls, a heart monitor implanted in a human, a smart meter that reports energy usage, mobile apps that can track your movement and location, and smart thermostats that adjust room temperatures based on weather forecasts and activity in the home.

1.13 Some Key Software Development Terminology

Figure 1.33 lists a number of buzzwords that you'll hear in the software development community.

Technology	Description
Agile software development	**Agile software development** is a set of methodologies that try to get software implemented faster and using fewer resources. Check out the Agile Alliance (`www.agilealliance.org`) and the Agile Manifesto (`www.agilemanifesto.org`).
Refactoring	**Refactoring** involves reworking programs to make them clearer and easier to maintain while preserving their correctness and functionality. It's widely employed with agile development methodologies. Many IDEs contain built-in *refactoring tools* to do major portions of the reworking automatically.
Design patterns	**Design patterns** are proven architectures for constructing flexible and maintainable object-oriented software. The field of design patterns tries to enumerate those recurring patterns, encouraging software designers to *reuse* them to develop better-quality software using less time, money and effort.
LAMP	**LAMP** is an acronym for the open-source technologies that many developers use to build web applications inexpensively—it stands for *Linux*, *Apache*, *MySQL* and *PHP* (or *Perl* or *Python*—two other popular scripting languages). MySQL is an open-source database-management system. PHP is a popular open-source server-side "scripting" language for developing web applications. Apache is the most popular web server software. The equivalent for Windows development is WAMP—*Windows*, *Apache*, *MySQL* and *PHP*.

Fig. 1.33 | Software technologies. (Part 1 of 2.)

Technology	Description
Software as a Service (SaaS)	Software has generally been viewed as a product; most software still is offered this way. If you want to run an application, you buy a software package from a software vendor—often a CD, DVD or web download. You then install that software on your computer and run it as needed. As new versions appear, you upgrade your software, often at considerable cost in time and money. This process can become cumbersome for organizations that must maintain tens of thousands of systems on a diverse array of computer equipment. With **Software as a Service (SaaS)**, the software runs on servers elsewhere on the Internet. When that server is updated, all clients worldwide see the new capabilities—no local installation is needed. You access the service through a browser. Browsers are quite portable, so you can run the same applications on a wide variety of computers from anywhere in the world. Salesforce.com, Google, Microsoft and many other companies offer SaaS.
Platform as a Service (PaaS)	**Platform as a Service (PaaS)** provides a computing platform for developing and running applications as a service over the web, rather than installing the tools on your computer. Some PaaS providers are Google App Engine, Amazon EC2 and Windows Azure™.
Cloud computing	SaaS and PaaS are examples of cloud computing. You can use software and data stored in the "cloud"—i.e., accessed on remote computers (or servers) via the Internet and available on demand—rather than having it stored locally on your desktop, notebook computer or mobile device. This allows you to increase or decrease computing resources to meet your needs at any given time, which is more cost effective than purchasing hardware to provide enough storage and processing power to meet occasional peak demands. Cloud computing also saves money by shifting to the service provider the burden of managing these apps (such as installing and upgrading the software, security, backups and disaster recovery).
Software Development Kit (SDK)	**Software Development Kits (SDKs)** include the tools and documentation developers use to program applications.

Fig. 1.33 | Software technologies. (Part 2 of 2.)

Software is complex. Large, real-world software applications can take many months or even years to design and implement. When large software products are under development, they typically are made available to the user communities as a series of releases, each more complete and polished than the last (Fig. 1.34).

Version	Description
Alpha	*Alpha* software is the earliest release of a software product that's still under active development. Alpha versions are often buggy, incomplete and unstable and are released to a relatively small number of developers for testing new features, getting early feedback, etc.
Beta	*Beta* versions are released to a larger number of developers later in the development process after most major bugs have been fixed and new features are nearly complete. Beta software is more stable, but still subject to change.
Release candidates	*Release candidates* are generally *feature complete*, (mostly) bug free and ready for use by the community, which provides a diverse testing environment—the software is used on different systems, with varying constraints and for a variety of purposes.
Final release	Any bugs that appear in the release candidate are corrected, and eventually the final product is released to the general public. Software companies often distribute incremental updates over the Internet.
Continuous beta	Software that's developed using this approach (for example, Google search or Gmail) generally does not have version numbers. It's hosted in the *cloud* (not installed on your computer) and is constantly evolving so that users always have the latest version.

Fig. 1.34 | Software product-release terminology.

1.14 C++11 and C++14: The Latest C++ Versions

11 C++11 was published by ISO/IEC in 2011. Bjarne Stroustrup, the creator of C++, expressed his vision for the future of the language—the main goals were to make C++ easier to learn, improve library-building capabilities and increase compatibility with the C programming language. C++11 extended the C++ Standard Library and added several features and enhancements to improve performance and security. The three compilers we use in this book

- Visual Studio 2015 Community Edition (Microsoft Windows)
- GNU C++ (Linux)
- Clang/LLVM in Xcode (Mac OS X)

have implemented most C++11 features.

14 The current C++ standard, C++14, was published by ISO/IEC in 2014. It added several language features and C++ Standard Library enhancements, and fixed bugs from C++11. Throughout this book, we cover features of C++11 and C++14 as appropriate for

a book at this level. For a list of C++11 and C++14 features and the compilers that support them, visit

```
http://en.cppreference.com/w/cpp/compiler_support
```

The next version of the C++ standard, C++17, is currently under development. For a list of proposed features, see

```
https://en.wikipedia.org/wiki/C%2B%2B17
```

17

1.15 Boost C++ Libraries

The **Boost C++ Libraries** (`www.boost.org`) are free, open-source libraries created by members of the C++ community. They are peer reviewed and portable across many compilers and platforms. Boost has grown to over 130 libraries, with more being added regularly. Today there are thousands of programmers in the Boost open-source community. The Boost libraries work well with the existing C++ Standard Library and often act as a proving ground for capabilities that are eventually absorbed into the C++ Standard Library. For example, the C++11 "regular expression" and "smart pointer" libraries, among others, are based on work done by the Boost community.

Regular expressions are used to match specific character patterns in text. They can be used to validate data to ensure that it's in a particular format, to replace parts of one string with another, or to split a string.

Many common bugs in C and C++ code are related to pointers, a powerful programming capability that C++ absorbed from C. As you'll see, **smart pointers** help you avoid some key errors associated with traditional pointers.

1.16 Keeping Up to Date with Information Technologies

Figure 1.35 lists key technical and business publications that will help you stay up-to-date with the latest news, trends and technology. You can also find a growing list of Internet- and web-related Resource Centers at `www.deitel.com/ResourceCenters.html`.

Publication	URL
AllThingsD	`allthingsd.com`
Bloomberg BusinessWeek	`www.businessweek.com`
CNET	`news.cnet.com`
Communications of the ACM	`cacm.acm.org`
Computerworld	`www.computerworld.com`
Engadget	`www.engadget.com`
eWeek	`www.eweek.com`
Fast Company	`www.fastcompany.com`
Fortune	`fortune.com`
GigaOM	`gigaom.com`

Fig. 1.35 | Technical and business publications. (Part 1 of 2.)

Publication	URL
Hacker News	news.ycombinator.com
IEEE Computer Magazine	www.computer.org/portal/web/computingnow/computer
InfoWorld	www.infoworld.com
Mashable	mashable.com
PCWorld	www.pcworld.com
SD Times	www.sdtimes.com
Slashdot	slashdot.org
Stack Overflow	stackoverflow.com
Technology Review	technologyreview.com
Techcrunch	techcrunch.com
The Next Web	thenextweb.com
The Verge	www.theverge.com
Wired	www.wired.com

Fig. 1.35 | Technical and business publications. (Part 2 of 2.)

Self-Review Exercises

1.1 Fill in the blanks in each of the following statements:
a) _____ devices use a network of satellites to retrieve location-based information.
b) Every year or two, the capacities of computers have approximately doubled inexpensively. This statement is known as _____.
c) A _____ is an electronic collection of data that's organized for easy access and manipulation.
d) Any computer can directly understand only its own _____.
e) Translator programs called _____ convert high-level language programs into machine language.
f) _____ allow you listen to large catalogues of music over the web, create customized "radio stations" and discover new music based on your feedback.
g) _____ are dropping dramatically, owing to rapid developments in hardware and software technologies.

1.2 Fill in the blanks in each of the following sentences about the C++ environment.
a) C++ source code filenames often end with the _____, _____, _____ or _____ extensions.
b) C++ offers software packages for Microsoft Windows such as _____.
c) In a C++ system, a(n) _____ program executes automatically before the compiler's translation phase begins.
d) There is also a standard error stream referred to as _____.

1.3 Fill in the blanks in each of the following statements (based on Section 1.8):
a) Objects have the property of _____—although objects may know how to communicate with one another across well-defined interfaces, they normally are not allowed to know how other objects are implemented.

b) C++ programmers concentrate on creating _____, which contain data members and the member functions that manipulate those data members and provide services to clients.
c) The process of analyzing and designing a system from an object-oriented point of view is called _____.
d) With _____, new classes of objects are derived by absorbing characteristics of existing classes, then adding unique characteristics of their own.
e) _____ is a graphical language that allows people who design software systems to use an industry-standard notation to represent them.
f) The size, shape, color and weight of an object are considered _____ of the object's class.

Answers to Self-Review Exercises

1.1 a) Global Positioning System (GPS). b) Moore's Law. c) database. d) machine language. e) compilers. f) Streaming music services. g) Computing costs.

1.2 a) `.cpp`, `.cxx`, `.cc`, `.C` b) Microsoft Visual C++ c) pre-processor d) `cerr`.

1.3 a) information hiding. b) classes. c) object-oriented analysis and design (OOAD). d) inheritance. e) The Unified Modeling Language (UML). f) attributes.

Exercises

1.4 Fill in the blanks in each of the following statements:
a) Motion and orientation information from an accelerometer and position data from a GPS device are examples of newer forms of _____ data.
b) The smallest data item that a computer can hold is a _____ which stands for _____ digit.
c) _____ is a type of computer language that uses English-like abbreviations for machine-language instructions.
d) _____ is a logical unit of the computer that sends information which has already been processed by the computer to various devices so that it may be used outside the computer.
e) _____ is a logical unit of the computer that retains information but is volatile (data is typically lost when the power is switched off) whereas _____ provides a means of retaining large chunks of information relatively permanently.
f) _____ is a logical unit of the computer that performs calculations and makes logical decisions.
g) The speed of a computer processor can be measured by the number of _____ it performs per _____.
h) _____ languages are most convenient to the programmer for writing programs quickly and easily.
i) The only language a computer can directly understand is that computer's _____.
j) _____ is a logical unit of the computer that coordinates the activities of all the other logical units.

1.5 Fill in the blanks in each of the following statements:
a) _____ are used to match specific character patterns in text.
b) _____ (formerly called C++0x)—the latest C++ programming language standard—was published by ISO/IEC in 2011.

1.6 Fill in the blanks in each of the following statements:
a) Software product-release categories are _____, _____, _____ are _____.
b) _____ involves reworking programs to make them clearer and easier to maintain while preserving their correctness and functionality.

1.7 You're probably wearing on your wrist one of the world's most common types of objects—a watch. Discuss how each of the following terms and concepts applies to the notion of a watch: object, attributes, behaviors, class, inheritance (consider, for example, an alarm clock), modeling, messages, encapsulation, interface and information hiding.

Making a Difference

Throughout the book we've included Making a Difference exercises in which you'll be asked to work on problems that really matter to individuals, communities, countries and the world.

1.8 ***(Test Drive: Carbon Footprint Calculator)*** Some scientists believe that carbon emissions, especially from the burning of fossil fuels, contribute significantly to global warming and that this can be combatted if individuals take steps to limit their use of carbon-based fuels. Various organizations and individuals are increasingly concerned about their "carbon footprints." Websites such as TerraPass

```
http://www.terrapass.com/carbon-footprint-calculator-2/
```

and Carbon Footprint

```
http://www.carbonfootprint.com/calculator.aspx
```

provide carbon footprint calculators. Test drive these calculators to determine your carbon footprint. Exercises in later chapters will ask you to program your own carbon footprint calculator. To prepare for this, research the formulas for calculating carbon footprints.

1.9 ***(Test Drive: Body Mass Index Calculator)*** By recent estimates, two-thirds of the people in the United States are overweight and about half of those are obese. This causes significant increases in illnesses such as diabetes and heart disease. To determine whether a person is overweight or obese, you can use a measure called the body mass index (BMI). The United States Department of Health and Human Services provides a BMI calculator at `http://www.nhlbi.nih.gov/guidelines/obesity/BMI/bmicalc.htm`. Use it to calculate your own BMI. An exercise in Chapter 2 will ask you to program your own BMI calculator. To prepare for this, research the formulas for calculating BMI.

1.10 ***(Attributes of Hybrid Vehicles)*** In this chapter you learned the basics of classes. Now you'll begin "fleshing out" aspects of a class called "Hybrid Vehicle." Hybrid vehicles are becoming increasingly popular, because they often get much better mileage than purely gasoline-powered vehicles. Browse the web and study the features of four or five of today's popular hybrid cars, then list as many of their hybrid-related attributes as you can. For example, common attributes include city-miles-per-gallon and highway-miles-per-gallon. Also list the attributes of the batteries (type, weight, etc.).

1.11 ***(Gender Neutrality)*** Some people want to eliminate sexism in all forms of communication. You've been asked to create a program that can process a paragraph of text and replace gender-specific words with gender-neutral ones. Assuming that you've been given a list of gender-specific words and their gender-neutral replacements (e.g., replace "wife" with "spouse," "man" with "person," "daughter" with "child" and so on), explain the procedure you'd use to read through a paragraph of text and manually perform these replacements. How might your procedure generate a strange term like "woperchild," which is actually listed in the Urban Dictionary (`www.urbandictionary.com`)? In Chapter 4, you'll learn that a more formal term for "procedure" is "algorithm," and that an algorithm specifies the steps to be performed and the order in which to perform them.

1.12 ***(Authentication)*** Suppose an authentication server used for email services to post all of the email correspondences for millions of people, including yours, on the Internet is being hacked by a group of disgruntled employees of the sever maintenance company. Discuss the issues.

1.13 ***(Programmer Responsibility and Liability)*** As a programmer, you may develop software that could affect people's social life and security. Suppose a software bug in one of your programs were to cause religious extremism and terrorism. Discuss the issues.

1.14 ***(Global world)*** The internet, especially after the advent of social media, is often given a bad name. Some see it as having no real value. One way to counter this argument is to discuss how the internet can be used to change our world for the better. Use the world wide web to investigate the problem of world hunger and discuss how the internet may be used to alleviate this issue.

Making a Difference Resources

The *Microsoft Imagine Cup* is a global competition in which students use technology to try to solve some of the world's most difficult problems, such as environmental sustainability, ending hunger, emergency response, literacy and more. For more information about the competition and to learn about previous winners' projects, visit `https://www.imaginecup.com/Custom/Index/About`. You can also find several project ideas submitted by worldwide charitable organizations. For additional ideas for programming projects that can make a difference, search the web for "making a difference" and visit the following websites:

`http://www.un.org/millenniumgoals`
The United Nations Millennium Project seeks solutions to major worldwide issues such as environmental sustainability, gender equality, child and maternal health, universal education and more.

`http://www.ibm.com/smarterplanet`
The IBM® Smarter Planet website discusses how IBM is using technology to solve issues related to business, cloud computing, education, sustainability and more.

`http://www.gatesfoundation.org`
The Bill and Melinda Gates Foundation provides grants to organizations that work to alleviate hunger, poverty and disease in developing countries.

`http://nethope.org`
NetHope is a collaboration of humanitarian organizations worldwide working to solve technology problems such as connectivity, emergency response and more.

`http://www.rainforestfoundation.org`
The Rainforest Foundation works to preserve rainforests and to protect the rights of the indigenous people who call the rainforests home. The site includes a list of things you can do to help.

`http://www.undp.org`
The United Nations Development Programme (UNDP) seeks solutions to global challenges such as crisis prevention and recovery, energy and the environment, democratic governance and more.

`http://www.unido.org`
The United Nations Industrial Development Organization (UNIDO) seeks to reduce poverty, give developing countries the opportunity to participate in global trade, and promote energy efficiency and sustainability.

`http://www.usaid.gov/`
USAID promotes global democracy, health, economic growth, conflict prevention, humanitarian aid and more.

`http://www.thp.org/knowledge-center/know-your-world-facts-about-hunger-poverty/ -`
The Hunger project is a global, non-profit aiming for a sustainable end to world hunger.

`https://www.wfp.org/hunger`
Part of the United Nations system, the World Food Programme in a humanitarian agency fighting world hunger.

`https://www.freedomfromhunger.org/about-us -`
Freedom from Hunger fights against chronic hunger and poverty, working with local partners to equip families with the resources needed to build a better future.

2

Introduction to C++ Programming, Input/Output and Operators

Objectives

In this chapter you'll:

- Write basic computer programs in C++.
- Write input and output statements.
- Use fundamental types.
- Learn computer memory concepts.
- Use arithmetic operators.
- Understand the precedence of arithmetic operators.
- Write decision-making statements.

Outline

2.1 Introduction

We now introduce C++ programming, which facilitates a disciplined approach to program development. Most of the C++ programs you'll study in this book process data and display results. In this chapter, we present five examples that demonstrate how your programs can display messages and obtain data from the user for processing. The first three examples display messages on the screen. The next obtains two numbers from a user at the keyboard, calculates their sum and displays the result. The accompanying discussion shows you how to perform *arithmetic calculations* and save their results for later use. The fifth example demonstrates *decision making* by showing you how to *compare* two numbers, then display messages based on the comparison results. We analyze each program one line at a time to help you ease into C++ programming.

Compiling and Running Programs

We've posted videos that demonstrate compiling and running programs in Microsoft Visual C++, GNU C++ and Xcode Clang/LLVM at

```
http://www.deitel.com/books/cpphtp10
```

2.2 First Program in C++: Printing a Line of Text

Consider a simple program that prints a line of text (Fig. 2.1). This program illustrates several important features of the C++ language. The text in lines 1–10 is the program's *source code* (or *code*). The line numbers are not part of the source code.

```
1  // Fig. 2.1: fig02_01.cpp
2  // Text-printing program.
3  #include <iostream> // enables program to output data to the screen
4
5  // function main begins program execution
6  int main() {
7     std::cout << "Welcome to C++!\n"; // display message
8
9     return 0; // indicate that program ended successfully
10 } // end function main
```

```
Welcome to C++!
```

Fig. 2.1 | Text-printing program.

Comments
Lines 1 and 2

```
// Fig. 2.1: fig02_01.cpp
// Text-printing program.
```

each begin with `//`, indicating that the remainder of each line is a **comment**. You insert comments to *document* your programs and to help other people read and understand them. Comments do not cause the computer to perform any action when the program is run—they're *ignored* by the C++ compiler and do *not* cause any machine-language object code to be generated. The comment `Text-printing program` describes the purpose of the program. A comment beginning with `//` is called a **single-line comment** because it terminates at the end of the current line. You also may use comments containing one or more lines enclosed in `/*` and `*/`, as in

```
/* Fig. 2.1: fig02_01.cpp
   Text-printing program. */
```

Good Programming Practice 2.1
Every program should begin with a comment that describes the purpose of the program.

`#include` Preprocessing Directive
Line 3

```
#include <iostream> // enables program to output data to the screen
```

is a **preprocessing directive**, which is a message to the C++ preprocessor (introduced in Section 1.9). Lines that begin with `#` are processed by the preprocessor *before* the program is compiled. This line notifies the preprocessor to include in the program the contents of the **input/output stream header `<iostream>`**. This header is a file containing information the compiler uses when compiling any program that outputs data to the screen or inputs data from the keyboard using C++'s stream input/output. The program in Fig. 2.1 outputs data to the screen, as we'll soon see. We discuss headers in more detail in Chapter 6 and explain the contents of `<iostream>` in Chapter 13.

Common Programming Error 2.1
Forgetting to include the `<iostream>` header in a program that inputs data from the keyboard or outputs data to the screen causes the compiler to issue an error message.

Blank Lines and White Space
Line 4 is simply a *blank line*. You use blank lines, *space characters* and *tab characters* (i.e., "tabs") to make programs easier to read. Together, these characters are known as **white space**. White-space characters are normally *ignored* by the compiler.

The `main` Function
Line 5

```
// function main begins program execution
```

is a single-line comment indicating that program execution begins at the next line.

Line 6

```
int main() {
```

is a part of every C++ program. The parentheses after `main` indicate that **`main`** is a program building block called a **function**. C++ programs typically consist of one or more functions and classes (as you'll learn in Chapter 3). Exactly *one* function in every program *must* be named `main`. Figure 2.1 contains only one function. C++ programs begin executing at function `main`, even if `main` is not the first function defined in the program. The keyword `int` to the left of `main` indicates that `main` "returns" an integer (whole number) value. A **keyword** is a word in code that is reserved by C++ for a specific use. The complete list of C++ keywords can be found in Fig. 4.3. We'll explain what it means for a function to "return a value" when we demonstrate how to create your own functions in Section 3.3. For now, simply include the keyword `int` to the left of `main` in each of your programs.

The **left brace**, **`{`**, (end of line 6) must *begin* the **body** of every function. A corresponding **right brace**, **`}`**, (line 10) must *end* each function's body.

An Output Statement

Line 7

```
    std::cout << "Welcome to C++!\n"; // display message
```

instructs the computer to **perform an action**—namely, to print the characters contained between the double quotation marks. Together, the quotation marks and the characters between them are called a **string**, a **character string** or a **string literal**. In this book, we refer to characters between double quotation marks simply as strings. White-space characters in strings are *not* ignored by the compiler.

The entire line 7, including `std::cout`, the **`<<` operator**, the string `"Welcome to C++!\n"` and the **semicolon** (**`;`**), is called a **statement**. Most C++ statements end with a semicolon, also known as the **statement terminator** (we'll see some exceptions to this soon). Preprocessing directives (such as `#include`) do not end with a semicolon. Typically, output and input in C++ are accomplished with **streams** of data. Thus, when the preceding statement is executed, it sends the stream of characters `Welcome to C++!\n` to the **standard output stream object**—**`std::cout`**—which is normally "connected" to the screen.

Common Programming Error 2.2

Omitting the semicolon at the end of a C++ statement is a syntax error. The syntax of a programming language specifies the rules for creating proper programs in that language. A syntax error occurs when the compiler encounters code that violates C++'s language rules (i.e., its syntax). The compiler normally issues an error message to help you locate and fix the incorrect code. Syntax errors are also called compiler errors, compile-time errors or compilation errors, because the compiler detects them during the compilation phase. You cannot execute your program until you correct all the syntax errors in it. As you'll see, some compilation errors are not syntax errors.

Good Programming Practice 2.2

Indent the body of each function one level within the braces that delimit the function's body. This makes a program's functional structure stand out, making the program easier to read.

Good Programming Practice 2.3

Set a convention for the size of indent you prefer, then apply it uniformly. The tab key may be used to create indents, but tab stops may vary. We prefer three spaces per level of indent.

The `std` Namespace

The `std::` before `cout` is required when we use names that we've brought into the program by the preprocessing directive `#include <iostream>`. The notation `std::cout` specifies that we are using a name, in this case `cout`, that belongs to namespace `std`. The names `cin` (the standard input stream) and `cerr` (the standard error stream)—introduced in Chapter 1—also belong to namespace `std`. Namespaces are an advanced C++ feature that we discuss in depth in Chapter 23, Other Topics. For now, you should simply remember to include `std::` before each mention of `cout`, `cin` and `cerr` in a program. This can be cumbersome—we'll soon introduce `using` declarations and the `using` directive, which will enable you to omit `std::` before each use of a name in the `std` namespace.

The Stream Insertion Operator and Escape Sequences

In the context of an output statement, the `<<` operator is referred to as the **stream insertion operator**. When this program executes, the value to the operator's right, the right **operand**, is inserted in the output stream. Notice that the `<<` operator points toward where the data goes. A string literal's characters *normally* print exactly as they appear between the double quotes. However, the characters `\n` are *not* printed on the screen (Fig. 2.1). The backslash (`\`) is called an **escape character**. It indicates that a "special" character is to be output. When a backslash is encountered in a string of characters, the next character is combined with the backslash to form an **escape sequence**. The escape sequence `\n` means **newline**. It causes the **cursor** (i.e., the current screen-position indicator) to move to the beginning of the next line on the screen. Some common escape sequences are listed in Fig. 2.2.

Escape sequence	Description
`\n`	Newline. Position the screen cursor to the beginning of the next line.
`\t`	Horizontal tab. Move the screen cursor to the next tab stop.
`\r`	Carriage return. Position the screen cursor to the beginning of the current line; do not advance to the next line.
`\a`	Alert. Sound the system bell.
`\\`	Backslash. Used to print a backslash character.
`\'`	Single quote. Used to print a single-quote character.
`\"`	Double quote. Used to print a double-quote character.

Fig. 2.2 | Escape sequences.

The `return` Statement

Line 9

```
   return 0; // indicate that program ended successfully
```

is one of several means we'll use to **exit a function**. When the **`return` statement** is used at the end of `main`, as shown here, the value `0` indicates that the program has *terminated successfully*. The right brace, `}`, (line 10) indicates the end of function `main`. According to the C++ standard, if program execution reaches the end of `main` without encountering a `return` statement, it's assumed that the program terminated successfully—exactly as when the last statement in `main` is a `return` statement with the value `0`. For that reason, we *omit* the `return` statement at the end of `main` in subsequent programs.

A Note About Comments

As you write a new program or modify an existing one, you should *keep your comments up-to-date* with the program's code. You'll *often* need to make changes to existing programs—for example, to fix errors (commonly called *bugs*) that prevent a program from working correctly or to enhance a program. Updating your comments as you make code changes helps ensure that the comments accurately reflect what the code does. This will make your programs easier for you and others to understand and modify in the future.

2.3 Modifying Our First C++ Program

We now present two examples that modify the program of Fig. 2.1 to print text on one line by using multiple statements and to print text on several lines by using a single statement.

Printing a Single Line of Text with Multiple Statements

`Welcome to C++!` can be printed several ways. For example, Fig. 2.3 performs stream insertion in multiple statements (lines 7–8), yet produces the same output as the program of Fig. 2.1. [*Note:* From this point forward, we use a *colored background* to highlight the key features each program introduces.] Each stream insertion resumes printing where the previous one stopped. The first stream insertion (line 7) prints `Welcome` followed by a space, and because this string did not end with `\n`, the second stream insertion (line 8) begins printing on the *same* line immediately following the space.

```
1  // Fig. 2.3: fig02_03.cpp
2  // Printing a line of text with multiple statements.
3  #include <iostream> // enables program to output data to the screen
4
5  // function main begins program execution
6  int main() {
7     std::cout << "Welcome ";
8     std::cout << "to C++!\n";
9  } // end function main
```

```
Welcome to C++!
```

Fig. 2.3 | Printing a line of text with multiple statements.

Printing Multiple Lines of Text with a Single Statement

A single statement can print multiple lines by using newline characters, as in line 7 of Fig. 2.4. Each time the `\n` (newline) escape sequence is encountered in the output stream, the screen cursor is positioned to the beginning of the next line. To get a blank line in your output, place two newline characters back to back, as in line 7.

```
1  // Fig. 2.4: fig02_04.cpp
2  // Printing multiple lines of text with a single statement.
3  #include <iostream> // enables program to output data to the screen
4
5  // function main begins program execution
6  int main() {
7     std::cout << "Welcome\nto\n\nC++!\n";
8  } // end function main
```

```
Welcome
to

C++!
```

Fig. 2.4 | Printing multiple lines of text with a single statement.

2.4 Another C++ Program: Adding Integers

Our next program obtains two integers typed by a user at the keyboard, computes their sum and outputs the result using `std::cout`. Figure 2.5 shows the program and sample inputs and outputs. In the sample execution, we highlight the user's input in bold. The program begins execution with function `main` (line 6). The left brace (line 6) begins `main`'s body and the corresponding right brace (line 21) ends it.

```
1  // Fig. 2.5: fig02_05.cpp
2  // Addition program that displays the sum of two integers.
3  #include <iostream> // enables program to perform input and output
4
5  // function main begins program execution
6  int main() {
7     // declaring and initializing variables
8     int number1{0}; // first integer to add (initialized to 0)
9     int number2{0}; // second integer to add (initialized to 0)
10    int sum{0}; // sum of number1 and number2 (initialized to 0)
11
12    std::cout << "Enter first integer: "; // prompt user for data
13    std::cin >> number1; // read first integer from user into number1
14
15    std::cout << "Enter second integer: "; // prompt user for data
16    std::cin >> number2; // read second integer from user into number2
17
18    sum = number1 + number2; // add the numbers; store result in sum
19
20    std::cout << "Sum is " << sum << std::endl; // display sum; end line
21 } // end function main
```

```
Enter first integer: 45
Enter second integer: 72
Sum is 117
```

Fig. 2.5 | Addition program that displays the sum of two integers.

Variable Declarations and List Initialization

Lines 8–10

```
int number1{0}; // first integer to add (initialized to 0)
int number2{0}; // second integer to add (initialized to 0)
int sum{0}; // sum of number1 and number2 (initialized to 0)
```

are **declarations**. The identifiers `number1`, `number2` and `sum` are the names of **variables**. A variable is a location in the computer's memory where a value can be stored for use by a program. These declarations specify that the variables `number1`, `number2` and `sum` are data of type `int`, meaning that these variables will hold **integer** (whole number) values, such as 7, –11, 0 and 31914.

Lines 8–10 also *initialize* each variable to `0` by placing a value in braces (`{` and `}`) immediately following the variable's name—this is known as **list initialization**,[1] which was introduced in C++11. Previously, these declarations would have been written as:

```
int number1 = 0; // first integer to add (initialized to 0)
int number2 = 0; // second integer to add (initialized to 0)
int sum = 0; // sum of number1 and number2 (initialized to 0)
```

Error-Prevention Tip 2.1

Although it's not always necessary to initialize every variable explicitly, doing so will help you avoid many kinds of problems.

All variables *must* be declared with a name and a data type *before* they can be used in a program. Several variables of the same type may be declared in one declaration—for example, we could have declared and initialized all three variables in one declaration by using a **comma-separated list** as follows:

```
int number1{0}, number2{0}, sum{0};
```

This makes the program less readable and prevents us from providing comments that describe each variable's purpose.

Good Programming Practice 2.4

Declare only one variable in each declaration and provide a comment that explains the variable's purpose in the program.

Fundamental Types

We'll soon discuss the type `double` for specifying *real numbers* and the type `char` for specifying *character data*. Real numbers are numbers with decimal points, such as 3.4, 0.0 and –11.19. A `char` variable may hold only a single lowercase letter, uppercase letter, digit or special character (e.g., `$` or `*`). Types such as `int`, `double` and `char` are called **fundamental types**. Fundamental-type names consist of one or more *keywords* and therefore *must* appear in all lowercase letters. Appendix C contains the complete list of fundamental types.

Identifiers

A variable name (such as `number1`) is any valid **identifier** that is *not* a keyword. An identifier is a series of characters consisting of letters, digits and underscores (_) that does *not*

1. List initialization is also known as uniform initialization.

begin with a digit. C++ is **case sensitive**—uppercase and lowercase letters are *different*, so `a1` and `A1` are *different* identifiers.

Portability Tip 2.1

C++ allows identifiers of any length, but your C++ implementation may restrict identifier lengths. Use identifiers of 31 characters or fewer to ensure portability (and readability).

Good Programming Practice 2.5

Choosing meaningful identifiers helps make a program self-documenting—a person can understand the program simply by reading it rather than having to refer to program comments or documentation.

Good Programming Practice 2.6

Avoid using abbreviations in identifiers. This improves program readability.

Good Programming Practice 2.7

Do not use identifiers that begin with underscores and double underscores, because C++ compilers use names like that for their own purposes internally.

Placement of Variable Declarations

Declarations of variables can be placed almost anywhere in a program, but they *must* appear *before* their corresponding variables are used in the program. For example, in the program of Fig. 2.5, the declaration in line 8

```
int number1{0}; // first integer to add (initialized to 0)
```

could have been placed immediately before line 13

```
std::cin >> number1; // read first integer from user into number1
```

the declaration in line 9

```
int number2{0}; // second integer to add (initialized to 0)
```

could have been placed immediately before line 16

```
std::cin >> number2; // read second integer from user into number2
```

and the declaration in line 10

```
int sum{0}; // sum of number1 and number2 (initialized to 0)
```

could have been placed immediately before line 18

```
sum = number1 + number2; // add the numbers; store result in sum
```

Obtaining the First Value from the User

Line 12

```
std::cout << "Enter first integer: "; // prompt user for data
```

displays `Enter first integer:` followed by a space. This message is called a **prompt** because it directs the user to take a specific action. We like to pronounce the preceding statement as "`std::cout` *gets* the string `"Enter first integer: "`." Line 13

```
std::cin >> number1; // read first integer from user into number1
```

uses the **standard input stream object `cin`** (of namespace `std`) and the **stream extraction operator**, **`>>`**, to obtain a value from the keyboard. Using the stream extraction operator with `std::cin` takes character input from the standard input stream, which is usually the keyboard. We like to pronounce the preceding statement as, "`std::cin` *gives* a value to `number1`" or simply "`std::cin` *gives* `number1`."

When the computer executes the preceding statement, it waits for the user to enter a value for variable `number1`. The user responds by typing an integer (as characters), then pressing the *Enter* key (sometimes called the *Return* key) to send the characters to the computer. The computer converts the character representation of the number to an integer and assigns (i.e., copies) this number (or **value**) to the variable `number1`. Any subsequent references to `number1` in this program will use this same value. Pressing *Enter* also causes the cursor to move to the beginning of the next line on the screen.

Users can, of course, enter *invalid* data from the keyboard. For example, when your program is expecting the user to enter an integer, the user could enter alphabetic characters, special symbols (like # or @) or a number with a decimal point (like 73.5), among others. In these early programs, we assume that the user enters *valid* data. As you progress through the book, you'll learn how to deal with data-entry problems.

Obtaining the Second Value from the User

Line 15

```
std::cout << "Enter second integer: "; // prompt user for data
```

prints `Enter second integer:` on the screen, prompting the user to take action. Line 16

```
std::cin >> number2; // read second integer from user into number2
```

obtains a value for variable `number2` from the user.

Calculating the Sum of the Values Input by the User

The assignment statement in line 18

```
sum = number1 + number2; // add the numbers; store result in sum
```

adds the values of variables `number1` and `number2` and assigns the result to variable `sum` using the **assignment operator `=`**. We like to read this statement as, "`sum` *gets* the value of `number1` + `number2`." Most calculations are performed in assignment statements. The `=` operator and the `+` operator are called **binary operators** because each has *two* operands. In the case of the `+` operator, the two operands are `number1` and `number2`. In the case of the preceding `=` operator, the two operands are `sum` and the value of the expression `number1 + number2`.

Good Programming Practice 2.8

Place spaces on either side of a binary operator. This makes the operator stand out and makes the program more readable.

Displaying the Result

Line 20

```
std::cout << "Sum is " << sum << std::endl; // display sum; end line
```

displays the character string `Sum is` followed by the numerical value of variable `sum` followed by `std::endl`—a so-called **stream manipulator**. The name `endl` is an abbreviation for "end line" and belongs to namespace `std`. The `std::endl` stream manipulator outputs a newline, then "flushes the output buffer." This simply means that, on some systems where outputs accumulate in the machine until there are enough to "make it worthwhile" to display them on the screen, `std::endl` forces any accumulated outputs to be displayed at that moment. This can be important when the outputs are prompting the user for an action, such as entering data.

The preceding statement outputs multiple values of different types. The stream insertion operator "knows" how to output each type of data. Using multiple stream insertion operators (<<) in a single statement is referred to as **concatenating**, **chaining** or **cascading stream insertion operations**.

Calculations can also be performed in output statements. We could have combined the statements in lines 18 and 20 into the statement

```
std::cout << "Sum is " << number1 + number2 << std::endl;
```

thus eliminating the need for the variable `sum`.

A powerful feature of C++ is that you can create your own data types called classes (we discuss this capability in Chapter 3 and explore it in depth in Chapter 9). You can then "teach" C++ how to input and output values of these new data types using the >> and << operators (this is called **operator overloading**—a topic we explore in Chapter 10).

2.5 Memory Concepts

Variable names such as `number1`, `number2` and `sum` actually correspond to **locations** in the computer's memory. Every variable has a *name*, a *type*, a *size* and a *value*.

In the addition program of Fig. 2.5, when the statement in line 13

```
std::cin >> number1; // read first integer from user into number1
```

is executed, the integer typed by the user is placed into a memory location to which the name `number1` has been assigned by the compiler. Suppose the user enters `45` for `number1`. The computer will place `45` into the location `number1`, as shown in Fig. 2.6. When a value is placed in a memory location, the value *overwrites* the previous value in that location; thus, placing a new value into a memory location is said to be a **destructive** operation.

Fig. 2.6 | Memory location showing the name and value of variable `number1`.

Returning to our addition program, suppose the user enters `72` when the statement

```
std::cin >> number2; // read second integer from user into number2
```

is executed. This value is placed into the location `number2`, and memory appears as in Fig. 2.7. The variables' locations are not necessarily adjacent in memory.

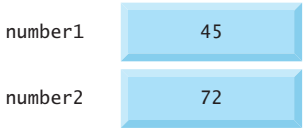


Fig. 2.7 | Memory locations after storing values in the variables for `number1` and `number2`.

Once the program has obtained values for `number1` and `number2`, it adds these values and places the total into the variable `sum`. The statement

```
sum = number1 + number2; // add the numbers; store result in sum
```

replaces whatever value was stored in `sum`. The calculated sum of `number1` and `number2` is placed into variable `sum` without regard to what value may already be in `sum`—that value is *lost*. After `sum` is calculated, memory appears as in Fig. 2.8. The values of `number1` and `number2` appear exactly as they did before the calculation. These values were used, but *not* destroyed, as the computer performed the calculation. Thus, when a value is read *out* of a memory location, the operation is **nondestructive**.

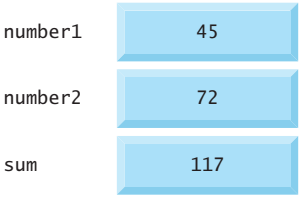


Fig. 2.8 | Memory locations after calculating and storing the `sum` of `number1` and `number2`.

2.6 Arithmetic

Most programs perform arithmetic calculations. Figure 2.9 summarizes the **arithmetic operators**. Note the use of various special symbols not used in algebra. The **asterisk** (`*`) indicates *multiplication* and the **percent sign** (`%`) is the remainder operator, which we'll discuss shortly. The arithmetic operators in Fig. 2.9 are all *binary* operators—they each take two operands. For example, the expression `number1 + number2` contains the binary operator `+` and the two operands `number1` and `number2`.

Operation	Arithmetic operator	Algebraic expression	C++ expression
Addition	`+`	$f + 7$	`f + 7`
Subtraction	`-`	$p - c$	`p - c`
Multiplication	`*`	bm or $b \cdot m$	`b * m`
Division	`/`	x / y or $\frac{x}{y}$ or $x \div y$	`x / y`
Remainder	`%`	$r \bmod s$	`r % s`

Fig. 2.9 | Arithmetic operators.

Integer division (i.e., where both the numerator and the denominator are integers) yields an integer quotient; for example, the expression `7 / 4` evaluates to `1` and the expression `17 / 5` evaluates to `3`. *Any fractional part in integer division is* ***truncated*** *(i.e., discarded)—no rounding occurs.*

The **remainder operator**, `%`, yields the *remainder after integer division* and can be used *only* with integer operands. The expression `x % y` yields the remainder after `x` is divided by `y`. Thus, `7 % 4` yields `3` and `17 % 5` yields `2`. In later chapters, we discuss interesting applications of the remainder operator, such as determining whether one number is a *multiple* of another (a special case of this is determining whether a number is *odd* or *even*).

Arithmetic Expressions in Straight-Line Form

Arithmetic expressions in C++ must be entered into the computer in **straight-line form**. Thus, expressions such as "a divided by b" must be written as `a / b`, so that all constants, variables and operators appear in a straight line. The algebraic notation

$$\frac{a}{b}$$

is generally *not* acceptable to compilers, although some special-purpose software packages do support more natural notation for complex mathematical expressions.

Parentheses for Grouping Subexpressions

Parentheses are used in C++ expressions in the same manner as in algebraic expressions. For example, to multiply `a` times the quantity `b + c` we write `a * (b + c)`.

Rules of Operator Precedence

C++ applies the operators in arithmetic expressions in a precise order determined by the following **rules of operator precedence**, which are generally the same as those in algebra:

1. Operators in expressions contained within pairs of *parentheses* are evaluated first. Parentheses are said to be at the "highest level of precedence." In cases of **nested**, or **embedded**, **parentheses**, such as

   ```
   (a * (b + c))
   ```

   the operators in the *innermost* pair of parentheses are applied first.

2. Multiplication, division and remainder operations are evaluated next. If an expression contains several multiplication, division and remainder operations, operators are applied from *left to right*. These three operators are said to be on the *same* level of precedence.

3. Addition and subtraction operations are applied last. If an expression contains several addition and subtraction operations, operators are applied from *left to right*. Addition and subtraction also have the *same* level of precedence.

The rules of operator precedence define the order in which C++ applies operators. When we say that certain operators are applied from left to right, we are referring to the **associativity** of the operators. For example, the addition operators (+) in the expression

```
a + b + c
```

associate from left to right, so `a + b` is calculated first, then `c` is added to that sum to determine the whole expression's value. We'll see that some operators associate from *right to left*.

Figure 2.10 summarizes these rules of operator precedence. We expand this table as we introduce additional C++ operators. Appendix A contains the complete precedence chart.

Operator(s)	Operation(s)	Order of evaluation (precedence)
`( )`	Parentheses	Evaluated first. For *nested* parentheses, such as in the expression `a * (b + c / (d + e))`, the expression in the *innermost* pair evaluates first. [*Caution:* If you have an expression such as `(a + b) * (c - d)` in which two sets of parentheses are not nested, but appear "on the same level," the C++ Standard does *not* specify the order in which these parenthesized subexpressions will evaluate.]

`*`
`/`
`%`
	Multiplication
	Division
	Remainder
		Evaluated second. If there are several, they're evaluated left to right.

`+` `-`	Addition Subtraction	Evaluated last. If there are several, they're evaluated left to right.

Fig. 2.10 | Precedence of arithmetic operators.

Sample Algebraic and C++ Expressions

Now consider several expressions in light of the rules of operator precedence. Each example lists an algebraic expression and its C++ equivalent. The following is an example of an arithmetic mean (average) of five terms:

Algebra: $m = \frac{a + b + c + d + e}{5}$

C++:
```
m = (a + b + c + d + e) / 5;
```

The parentheses are required because division has *higher* precedence than addition. The *entire* quantity `(a + b + c + d + e)` is to be divided by `5`. If the parentheses are erroneously omitted, we obtain `a + b + c + d + e / 5`, which evaluates incorrectly as

$$a + b + c + d + \frac{e}{5}$$

The following is an example of the equation of a straight line:

Algebra: $y = mx + b$

C++:
```
y = m * x + b;
```

No parentheses are required. The multiplication is applied first because multiplication has a *higher* precedence than addition.

The following example contains remainder (%), multiplication, division, addition, subtraction and assignment operations:

Algebra: $z = pr\%q + w/x - y$

C++:
```
z = p * r % q + w / x - y;
```

6 1 2 4 3 5

The circled numbers under the statement indicate the order in which C++ applies the operators. The multiplication, remainder and division operations are evaluated *first* in left-to-right order (i.e., they associate from left to right) because they have *higher precedence* than addition and subtraction. The addition and subtraction are applied next. These are also applied left to right. The assignment operator is applied *last* because its precedence is *lower* than that of any of the arithmetic operators.

Evaluation of a Second-Degree Polynomial

To develop a better understanding of the rules of operator precedence, consider the evaluation of a second-degree polynomial $y = ax^2 + bx + c$:

```
y = a * x * x + b * x + c;
    6   1   2   4   3   5
```

There is no arithmetic operator for exponentiation in C++, so we've represented x^2 as `x * x`. The circled numbers under the statement indicate the order in which C++ applies the operators. In Chapter 5, we'll discuss the standard library function `pow` ("power") that performs exponentiation.

Suppose variables `a`, `b`, `c` and `x` in the preceding second-degree polynomial are initialized as follows: `a = 2`, `b = 3`, `c = 7` and `x = 5`. Figure 2.11 illustrates the order in which the operators are applied and the final value of the expression.

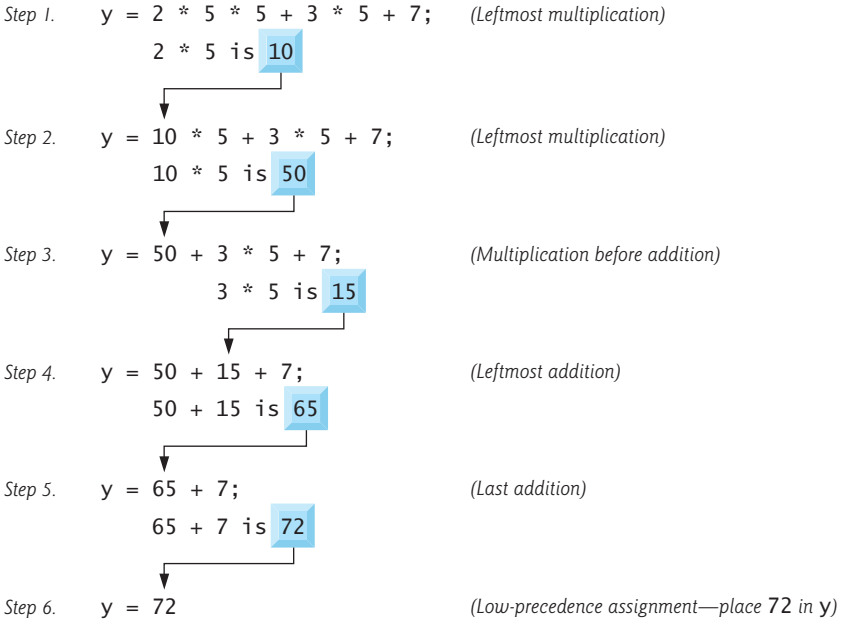


Fig. 2.11 | Order in which a second-degree polynomial is evaluated.

Redundant Parentheses

As in algebra, it's acceptable to place *unnecessary* parentheses in an expression to make it clearer. These are called **redundant parentheses**. For example, the second-degree polynomial could be parenthesized as follows:

```
y = (a * x * x) + (b * x) + c;
```

2.7 Decision Making: Equality and Relational Operators

We now introduce C++'s **`if` statement**, which allows a program to take alternative action based on whether a **condition** is true or false. Conditions in `if` statements can be formed by using the **relational operators** and **equality operators** summarized in Fig. 2.12. The relational operators all have the same level of precedence and associate left to right. The equality operators both have the same level of precedence, which is *lower* than that of the relational operators, and associate left to right.

Algebraic relational or equality operator	C++ relational or equality operator	Sample C++ condition	Meaning of C++ condition
Relational operators			
>	>	`x > y`	x is greater than y
<	<	`x < y`	x is less than y
≥	>=	`x >= y`	x is greater than or equal to y
≤	<=	`x <= y`	x is less than or equal to y
Equality operators			
=	==	`x == y`	x is equal to y
≠	!=	`x != y`	x is not equal to y

Fig. 2.12 | Relational and equality operators.

Common Programming Error 2.3

Reversing the order of the pair of symbols in the operators !=, >= and <= (by writing them as =!, => and =<, respectively) is normally a syntax error. In some cases, writing != as =! will not be a syntax error, but almost certainly will be a logic error that has an effect at execution time. You'll understand why when you learn about logical operators in Chapter 5. A fatal logic error causes a program to fail and terminate prematurely. A nonfatal logic error allows a program to continue executing, but usually produces incorrect results.

Common Programming Error 2.4

Confusing the equality operator == with the assignment operator = results in logic errors. We like to read the equality operator as "is equal to" or "double equals," and the assignment operator as "gets" or "gets the value of" or "is assigned the value of." As you'll see in Section 5.12, confusing these operators may not necessarily cause an easy-to-recognize syntax error, but may cause subtle logic errors.

Using the `if` *Statement*

The following example (Fig. 2.13) uses six `if` statements to compare two numbers input by the user. If a given `if` statement's condition is *true*, the output statement in the body of that `if` statement *executes*. If the condition is *false*, the output statement in the body *does not* execute.

```
1  // Fig. 2.13: fig02_13.cpp
2  // Comparing integers using if statements, relational operators
3  // and equality operators.
4  #include <iostream> // enables program to perform input and output
5
6  using std::cout; // program uses cout
7  using std::cin; // program uses cin
8  using std::endl; // program uses endl
9
10 // function main begins program execution
11 int main() {
12    int number1{0}; // first integer to compare (initialized to 0)
13    int number2{0}; // second integer to compare (initialized to 0)
14
15    cout << "Enter two integers to compare: "; // prompt user for data
16    cin >> number1 >> number2; // read two integers from user
17
18    if (number1 == number2) {
19       cout << number1 << " == " << number2 << endl;
20    }
21
22    if (number1 != number2) {
23       cout << number1 << " != " << number2 << endl;
24    }
25
26    if (number1 < number2) {
27       cout << number1 << " < " << number2 << endl;
28    }
29
30    if (number1 > number2) {
31       cout << number1 << " > " << number2 << endl;
32    }
33
34    if (number1 <= number2) {
35       cout << number1 << " <= " << number2 << endl;
36    }
37
38    if (number1 >= number2) {
39       cout << number1 << " >= " << number2 << endl;
40    }
41 } // end function main
```

Fig. 2.13 | Comparing integers using `if` statements, relational operators and equality operators. (Part 1 of 2.)

```
Enter two integers to compare: 3 7
3 != 7
3 < 7
3 <= 7
```

```
Enter two integers to compare: 22 12
22 != 12
22 > 12
22 >= 12
```

```
Enter two integers to compare: 7 7
7 == 7
7 <= 7
7 >= 7
```

Fig. 2.13 | Comparing integers using `if` statements, relational operators and equality operators. (Part 2 of 2.)

`using` Declarations

Lines 6–8

```
using std::cout; // program uses cout
using std::cin; // program uses cin
using std::endl; // program uses endl
```

are **`using` declarations** that eliminate the need to repeat the `std::` prefix as we did in earlier programs. We can now write `cout` instead of `std::cout`, `cin` instead of `std::cin` and `endl` instead of `std::endl`, respectively, in the remainder of the program.

In place of lines 6–8, many programmers prefer to provide the **`using` directive**

```
using namespace std;
```

which enables a program to use *all* the names in any standard C++ header (such as `<iostream>`) that a program might include. From this point forward in the book, we'll use the preceding directive in our programs.[2]

Variable Declarations and Reading the Inputs from the User

Lines 12–13

```
int number1{0}; // first integer to compare (initialized to 0)
int number2{0}; // second integer to compare (initialized to 0)
```

declare the variables used in the program and initialize them to `0`.

Line 16

```
cin >> number1 >> number2; // read two integers from user
```

2. In Chapter 23, Other Topics, we'll discuss some issues with `using` directives in large-scale systems.

uses *cascaded* stream extraction operations to input two integers. Recall that we're allowed to write `cin` (instead of `std::cin`) because of line 7. First a value is read into variable `number1`, then a value is read into variable `number2`.

Comparing Numbers

The `if` statement in lines 18–20

```
if (number1 == number2) {
   cout << number1 << " == " << number2 << endl;
}
```

compares the values of variables `number1` and `number2` to test for equality. If the values are equal, the statement in line 19 displays a line of text indicating that the numbers are equal. If the conditions are `true` in one or more of the `if` statements starting in lines 22, 26, 30, 34 and 38, the corresponding body statement displays an appropriate line of text.

Each `if` statement in Fig. 2.13 contains a single body statement that's indented. Also notice that we've enclosed each body statement in a pair of braces, `{ }`, creating what's called a **compound statement** or a **block**.

Good Programming Practice 2.9

Indent the statement(s) in the body of an `if` statement to enhance readability.

Error-Prevention Tip 2.2

You don't need to use braces, `{ }`, around single-statement bodies, but you must *include the braces around multiple-statement bodies. You'll see later that forgetting to enclose multiple-statement bodies in braces leads to errors. To avoid errors, as a rule, always enclose an `if` statement's body statement(s) in braces.*

Common Programming Error 2.5

Placing a semicolon immediately after the right parenthesis after the condition in an `if` statement is often a logic error (although not a syntax error). The semicolon causes the body of the `if` statement to be empty, so the `if` statement performs no action, regardless of whether or not its condition is true. Worse yet, the original body statement of the `if` statement now becomes a statement in sequence *with the `if` statement and* always *executes, often causing the program to produce incorrect results.*

White Space

Note our use of blank lines in Fig. 2.13. We inserted these for readability. Recall that white-space characters, such as tabs, newlines and spaces, are normally ignored by the compiler. So, statements may be split over several lines and may be spaced according to your preferences. It's a syntax error to split identifiers, strings (such as `"hello"`) and constants (such as the number `1000`) over several lines.

Good Programming Practice 2.10

A lengthy statement may be spread over several lines. If a statement must be split across lines, choose meaningful breaking points, such as after a comma in a comma-separated list, or after an operator in a lengthy expression. If a statement is split across two or more lines, indent all subsequent lines and left-align the group of indented lines.

Operator Precedence

Figure 2.14 shows the precedence and associativity of the operators introduced in this chapter. The operators are shown top to bottom in decreasing order of precedence. All these operators, with the exception of the assignment operator =, associate from left to right. Addition is left-associative, so an expression like `x + y + z` is evaluated as if it had been written `(x + y) + z`. The assignment operator = associates from *right to left*, so an expression such as `x = y = 0` is evaluated as if it had been written `x = (y = 0)`, which, as we'll soon see, first assigns `0` to `y`, then assigns the *result* of that assignment—`0`—to `x`.

Operators				Associativity	Type
`()`				*[See caution in Fig. 2.10]*	grouping parentheses
`*`	`/`	`%`		left to right	multiplicative
`+`	`-`			left to right	additive
`<<`	`>>`			left to right	stream insertion/extraction
`<`	`<=`	`>`	`>=`	left to right	relational
`==`	`!=`			left to right	equality
`=`				right to left	assignment

Fig. 2.14 | Precedence and associativity of the operators discussed so far.

Good Programming Practice 2.11

Refer to the operator precedence and associativity chart (Appendix A) when writing expressions containing many operators. Confirm that the operators in the expression are performed in the order you expect. If you're uncertain about the order of evaluation in a complex expression, break the expression into smaller statements or use parentheses to force the order of evaluation, exactly as you'd do in an algebraic expression. Be sure to observe that some operators such as assignment (=) associate right to left rather than left to right.

2.8 Wrap-Up

You learned many important basic features of C++ in this chapter, including displaying data on the screen, inputting data from the keyboard and declaring variables of fundamental types. In particular, you learned to use the output stream object `cout` and the input stream object `cin` to build simple interactive programs. We explained how variables are stored in and retrieved from memory. You also learned how to use arithmetic operators to perform calculations. We discussed the order in which C++ applies operators (i.e., the rules of operator precedence), as well as the associativity of the operators. You also learned how C++'s `if` statement allows a program to make decisions. Finally, we introduced the equality and relational operators, which we used to form conditions in `if` statements.

The non-object-oriented applications presented here introduced you to basic programming concepts. As you'll see in Chapter 3, C++ applications typically contain just a few lines of code in function `main`—these statements normally create the objects that perform the work of the application, then the objects "take over from there." In Chapter 3, you'll learn how to implement your own classes and use objects of those classes in applications.

Summary

Section 2.2 First Program in C++: Printing a Line of Text

- **Single-line comments** (p. 86) begin with `//`. You insert comments to document your programs and improve their readability.
- **Comments** do not cause the computer to perform any **action** (p. 87) when the program is run—they're ignored by the compiler.
- A **preprocessing directive** (p. 86) begins with `#` and is a message to the C++ preprocessor. Preprocessing directives are processed before the program is compiled.
- The line `#include <iostream>` (p. 86) tells the C++ preprocessor to include the contents of the input/output stream header, which contains information necessary to compile programs that output data to the screen or input data from the keyboard.
- **White space** (i.e., blank lines, space characters and tab characters; p. 86) makes programs easier to read. White-space characters outside of string literals are ignored by the compiler.
- C++ programs begin executing at `main` (p. 87), even if `main` does not appear first in the program.
- The keyword `int` to the left of `main` indicates that `main` "returns" an integer value.
- The **body** (p. 87) of every function must be contained in braces (`{` and `}`).
- A **string** (p. 87) in double quotes is sometimes referred to as a **character string**, **message** or **string literal**. White-space characters in strings are *not* ignored by the compiler.
- Most C++ **statements** (p. 87) end with a semicolon, also known as the statement terminator (we'll see some exceptions to this soon).
- Output and input in C++ are accomplished with **streams** (p. 87) of data.
- The output stream object `std::cout` (p. 87)—normally connected to the screen—is used to output data. Multiple data items can be output by concatenating **stream insertion** (`<<`; p. 88) operators.
- The input stream object `std::cin`—normally connected to the keyboard—is used to input data. Multiple data items can be input by concatenating stream extraction (`>>`) operators.
- The notation `std::cout` specifies that we are using `cout` from "namespace" `std`.
- When a backslash (i.e., an escape character) is encountered in a string of characters, the next character is combined with the backslash to form an **escape sequence** (p. 88).
- The **newline escape sequence** `\n` (p. 88) moves the cursor to the beginning of the next line on the screen.
- C++ keyword `return` (p. 89) is one of several means to exit a function.

Section 2.4 Another C++ Program: Adding Integers

- All **variables** (p. 91) in a C++ program must be declared before they can be used.
- Variables of type `int` (p. 91) hold integer values, i.e., whole numbers such as 7, –11, 0, 31914.
- A variable can be initialized in its declaration using **list initialization** (p. 91; introduced in C++11)—the variable's initial value is placed in braces (`{` and `}`) immediately following the variable's name.
- A variable name is any valid **identifier** (p. 91) that is not a keyword. An identifier is a series of characters consisting of letters, digits and underscores (_). Identifiers cannot start with a digit. Identifiers can be any length, but some systems or C++ implementations may impose length restrictions.
- A message that directs the user to take a specific action is known as a **prompt** (p. 92).

- C++ is **case sensitive** (p. 92).
- A program reads the user's input with the **`std::cin`** (p. 93) object and the **stream extraction** (>>; p. 93) operator.
- Most calculations are performed in **assignment statements** (p. 93).

Section 2.5 Memory Concepts

- A variable is a location in **memory** (p. 94) where a value can be stored for use by a program.
- Every variable stored in the computer's memory has a name, a value, a type and a size.
- Whenever a new value is placed in a memory location, the process is **destructive** (p. 94); i.e., the new value replaces the previous value in that location. The previous value is lost.
- When a value is read from memory, the process is **nondestructive** (p. 95); i.e., a copy of the value is read, leaving the original value undisturbed in the memory location.
- The **`std::endl` stream manipulator** (p. 94) outputs a newline, then "flushes the output buffer."

Section 2.6 Arithmetic

- C++ evaluates **arithmetic expressions** (p. 95) in a precise sequence determined by the **rules of operator precedence** (p. 96) and **associativity** (p. 96).
- Parentheses may be used to group expressions.
- **Integer division** (p. 96) yields an integer quotient. Any fractional part in integer division is truncated.
- The **remainder operator, `%`** (p. 96), yields the remainder after integer division.

Section 2.7 Decision Making: Equality and Relational Operators

- The **`if` statement** (p. 99) allows a program to take alternative action based on whether a condition is met. The format for an `if` statement is

```
if (condition) {
   statement;
}
```

  If the condition is true, the statement in the body of the `if` is executed. If the condition is not met, i.e., the condition is false, the body statement is skipped.
- Conditions in `if` statements are commonly formed by using **equality and relational operators** (p. 99). The result of using these operators is always the value *true* or *false*.
- The **`using` declaration** (p. 101)

```
using std::cout;
```

  informs the compiler where to find `cout` (namespace `std`) and eliminates the need to repeat the `std::` prefix. The **`using` directive** (p. 101)

```
using namespace std;
```

  enables the program to use all the names in any included C++ standard library header.

Self-Review Exercises

2.1 Fill in the blanks in each of the following.

a) Every C++ program begins execution at the function _____.

b) Lines that begin with _____ are processed by the preprocessor *before* the program is _____.

c) _____ indicates that the remainder of each line is a comment.

d) The equality operator is _____ while the assignment operator is _____.
e) A(n) _____ indicates the end of function main.

2.2 State whether each of the following is *true* or *false*. If *false*, explain why. (Assume the statement `using std::cout;` is used.):

a) Comments cause the computer to perform an action when the program is run and they're not *ignored* by the C++.
b) When a backslash is encountered in a string of characters, the next character is combined with the backslash to form an escape sequence.
c) C++ programs begin executing from the first encountered function.
d) All variables must be given a type when they're declared.
e) Exactly *one* function in every program *must* be named main.
f) A string literal's characters *normally* print exactly as they appear between the double quotes.
g) The return statement is used at the end of main.
h) The arithmetic operators `*`, `/`, `%`, `+` and `–` all have the same level of precedence.
i) Syntax errors are also called compiler errors, compile-time errors or compilation errors, because the compiler detects them during the compilation phase.

2.3 Write a single C++ statement to accomplish each of the following (assume that neither `using` declarations nor a `using` directive have been used):

a) Declare the variables `c`, `thisIsAVariable`, `q76354` and `number` to be of type `int` (in one statement) and initialize each to `0`.
b) Prompt the user to enter an integer. End your prompting message with a colon (`:`) followed by a space and leave the cursor positioned after the space.
c) Get a value from the user of integer type and print the value by adding 10 to it.
d) If the variable `value` is greater than 15, then subtract `5` from `value` and print the new `value`.
e) Print the message `"This is a C++ program"` on one line.
f) Print the message `"This is a C++ program"` on two lines. End the first line with `C++`.
g) Print the message `"This is a C++ program"` with each word on a separate line.
h) Print the message `"This is a C++ program"`. Separate each word from the next by a tab.

2.4 Write a statement (or comment) to accomplish each of the following (assume that `using` declarations have been used for `cin`, `cout` and `endl`):

a) Document that a program calculates the product of three integers.
b) Declare the variables `x`, `y`, `z` and `result` to be of type `int` (in separate statements) and initialize each to 0.
c) Prompt the user to enter three integers.
d) Read three integers from the keyboard and store them in the variables `x`, `y` and `z`.
e) Compute the product of the three integers contained in variables `x`, `y` and `z`, and assign the result to the variable `result`.
f) Print `"The product is "` followed by the value of the variable `result`.
g) Return a value from `main` indicating that the program terminated successfully.

2.5 Using the statements you wrote in Exercise 2.4, write a complete program that calculates and displays the product of three integers. Add comments to the code where appropriate. [*Note:* You'll need to write the necessary `using` declarations or directive.]

2.6 Identify and correct the errors in each of the following statements (assume that the statement `using std::cout;` is used):

a)
```
if ( c < 7
   cout << "c is less than 7\n";
```

b)
```
if ( c =! 7 )
   cout << "c is equal to or greater than 7\n";
```

Answers to Self-Review Exercises

2.1 a) `main`. b) #, compiled. c) //. d) ==, =. e) Right brace }.

2.2 a) *False*. They are ignored by the C++.
b) *True*.
c) *False*. The execute from the main function only.
d) *True*.
e) *True*.
f) *True*.
g) *True*.
h) *False*. The operators `*`, `/` and `%` have the same precedence, and the operators `+` and `-` have a lower precedence.
i) *True*.

2.3 a)
```
int c{0}, thisIsAVariable{0}, q76354{0}, number{0};
```
b)
```
std::cout << "Enter an integer: ";
```
c)
```
cin>> a; cout<< a+10;
```
d)
```
if (a>15)
   cout<< a-5;
else
   cout<< a;)
```
e)
```
std::cout << "This is a C++ program\n";
```
f)
```
std::cout << "This is a C++\nprogram\n";
```
g)
```
std::cout << "This\nis\na\nC++\nprogram\n";
```
h)
```
std::cout << "This\tis\ta\tC++\tprogram\n";
```

2.4 a)
```
// Calculate the product of three integers
```
b)
```
int x{0};
int y{0};
int z{0};
int result{0};
```
c)
```
cout << "Enter three integers: ";
```
d)
```
cin >> x >> y >> z;
```
e)
```
result = x * y * z;
```
f)
```
cout << "The product is " << result << endl;
```
g)
```
return 0;
```

2.5 (See program below.)

```
1  // Calculate the product of three integers
2  #include <iostream> // enables program to perform input and output
3  using namespace std; // program uses names from the std namespace
4
5  // function main begins program execution
6  int main() {
7     int x{0}; // first integer to multiply
8     int y{0}; // second integer to multiply
9     int z{0}; // third integer to multiply
10    int result{0}; // the product of the three integers
11
```